

Scaling Laws, Emergence, and In-context Learning

Aaron Mueller

CS599 B1: Advanced Natural Language Processing

Sep. 22, 2026

Administrivia

- A practice quiz has been released! See the course website or Piazza.
- **HW1** is due on Thursday, 11:59pm ET!
 - Remember that you have 5 free late days
 - Also remember that you don't need to ask me to use them
 - If you recently joined the course and aren't sure if you'll be able to catch up, please come to office hours

Scaling Laws

Distributional Semantics

Those of you who took CS505 may be familiar with the concept of **distributional semantics**:

“You shall know a word by the company it keeps.” —**Firth, 1957**



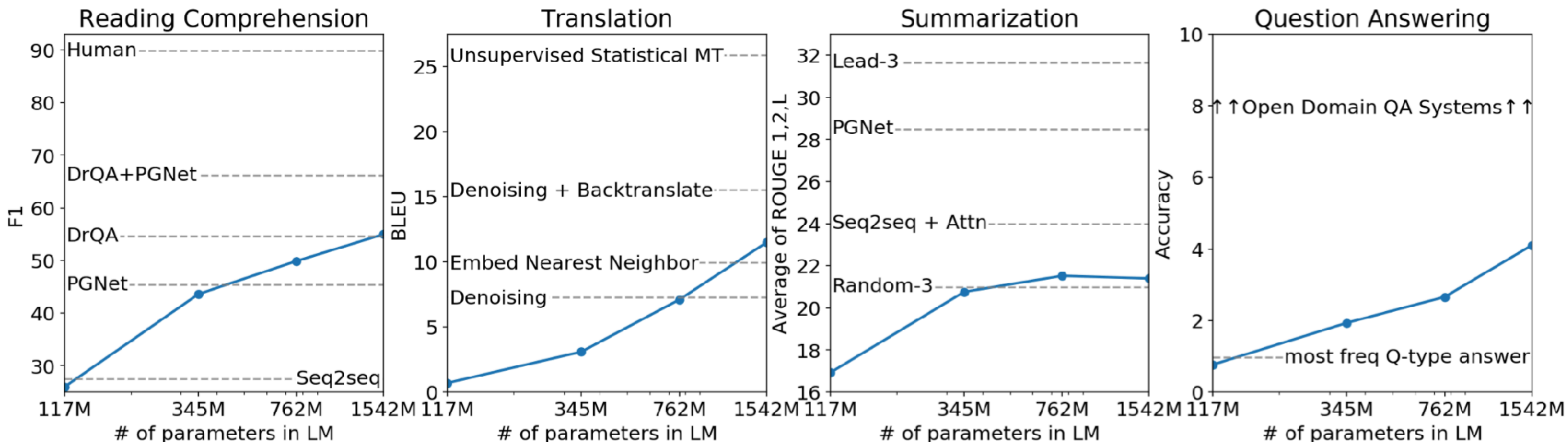
You can never fully characterize the distribution of a word: *more data will always give you more accurate estimates*.

- Also, word distributions are always changing because language is always changing!

Given this, it seems like more data will always be better. *This is empirically what we've observed over the last decade.*

The Unreasonable Effectiveness of Pretraining

[Radford et al., 2019]



As we train larger and larger models, performance on most NLP tasks seems to increase.

It turns out that there are *predictable* improvements as we increase dataset size and model size!

Hyperparameter Optimization

- For HW1, you're asked to tune hyperparameters. How do we typically do this?
- Define some grid of values:

Initial learning rate: {5e-3, 1e-3, 5e-4, 1e-4, 5e-5, 1e-5, ...}

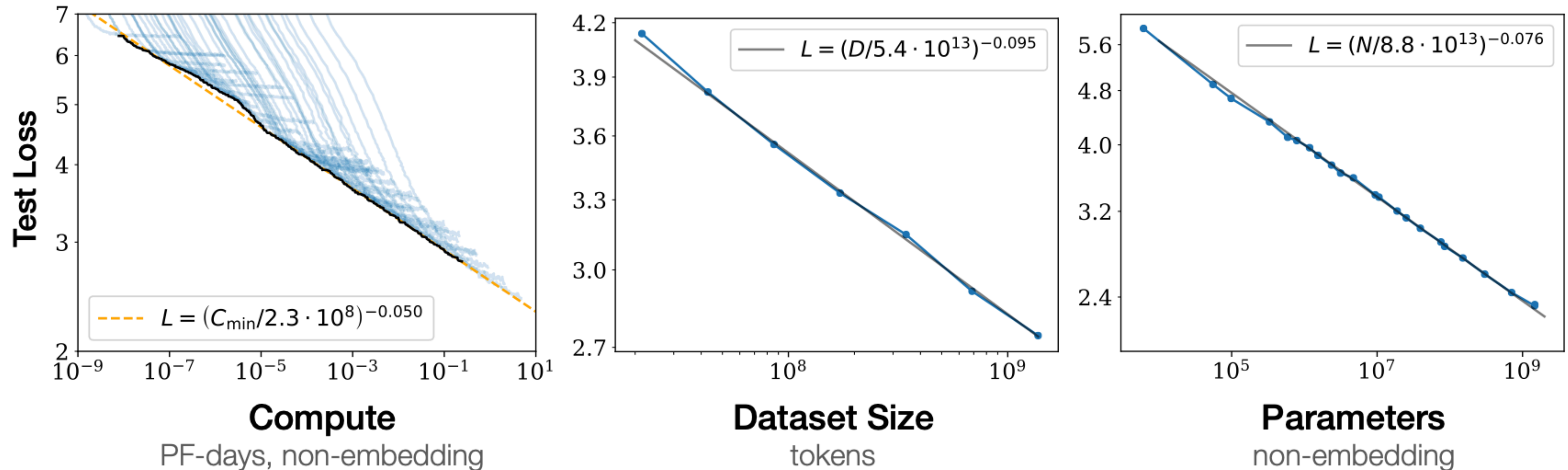
Batch size: {32, 64, 128, 256, ...}

Weight decay: {0, 0.1, 0.01, 0.001, ...}

- Loop over all combinations, pick whichever is best on some validation set
- This is expensive. What if we can only afford 1 to 5 full runs?

Scaling Laws

[Kaplan et al., 2020]



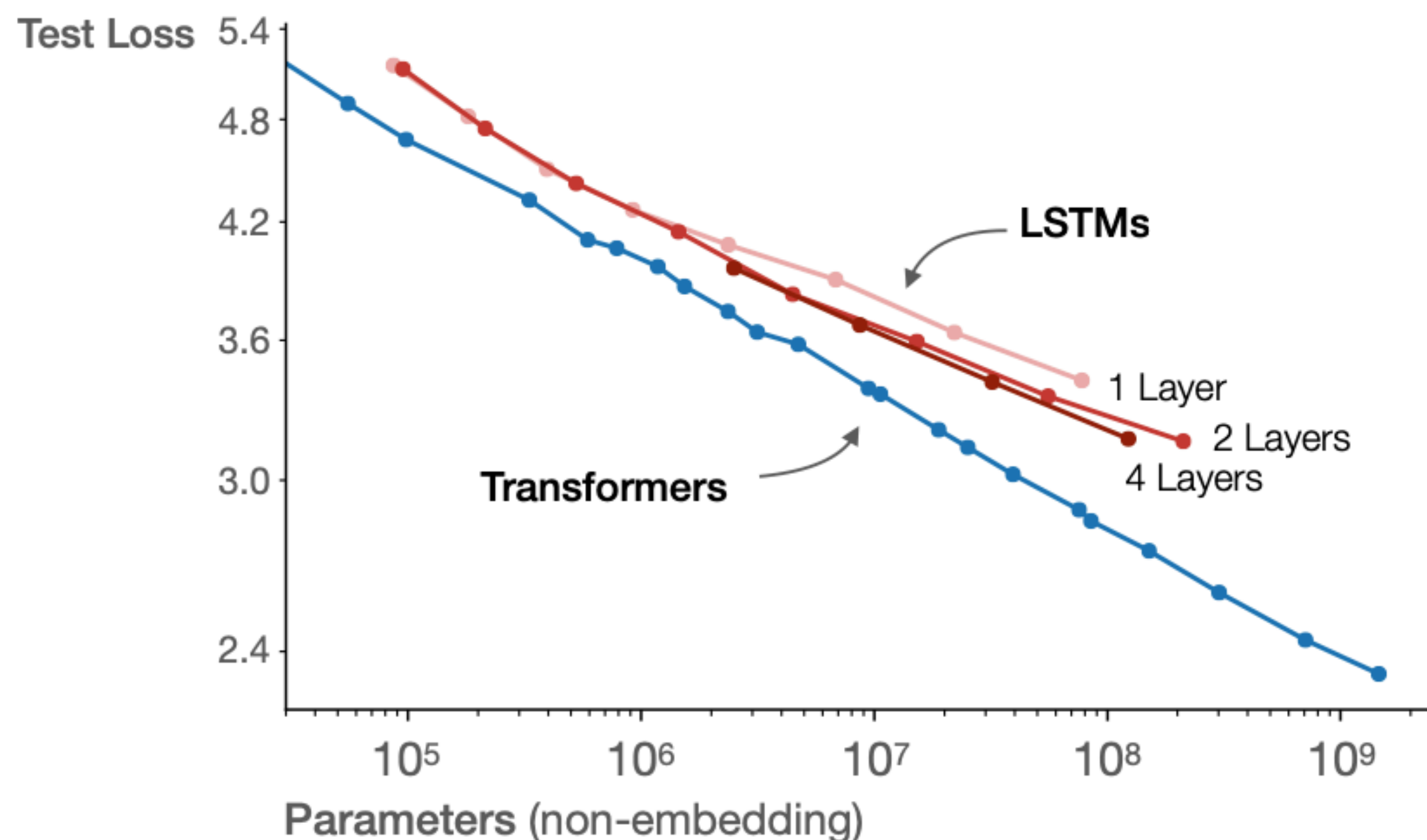
Scaling laws yield simple-yet-predictive rules for model performance w.r.t. increasing scale (of data and models).

Big implications: innovate at small scale, and extrapolate to larger versions of that model.

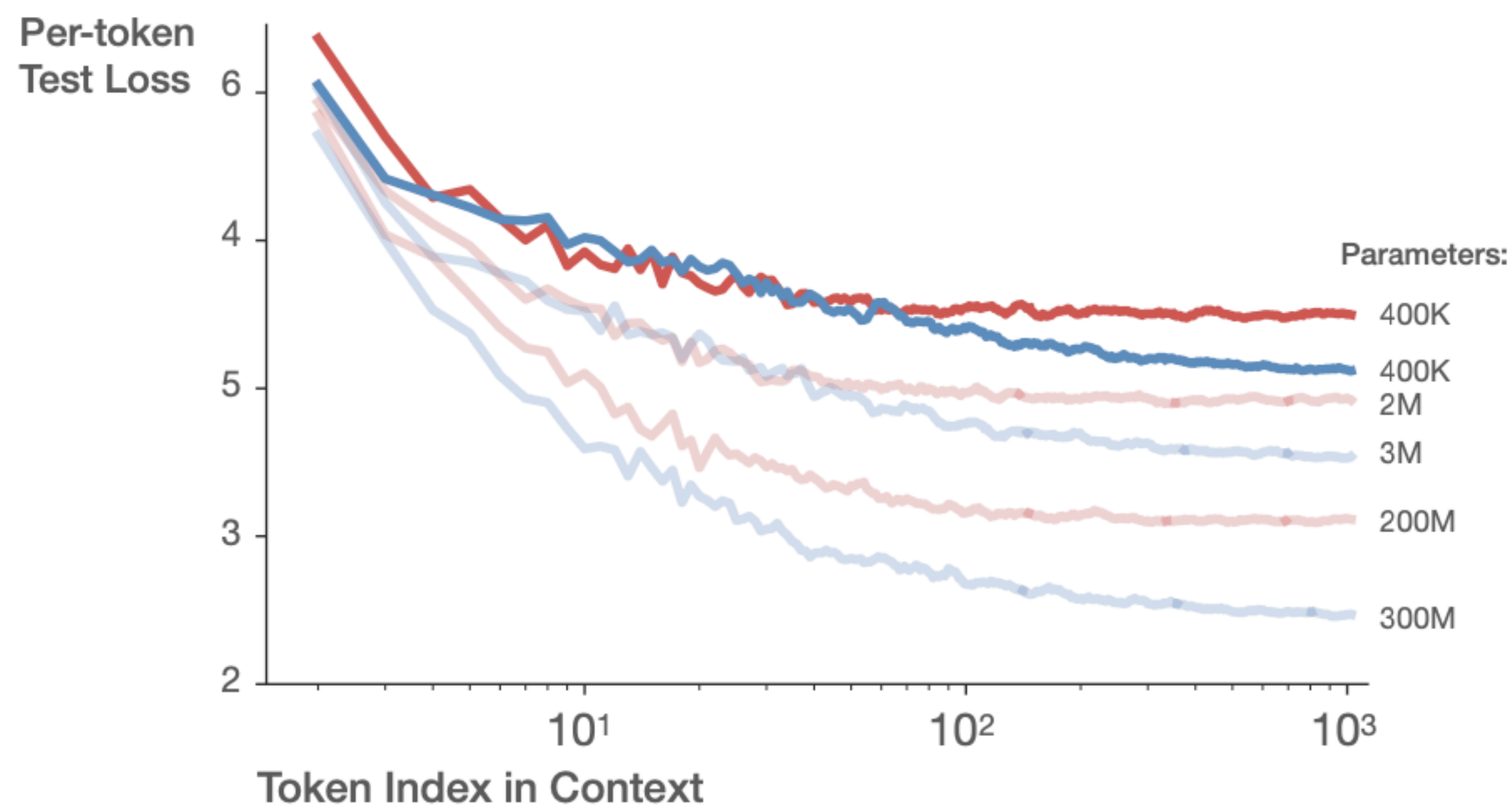
Scaling Laws

Architecture

Transformers asymptotically outperform LSTMs
due to improved use of long contexts

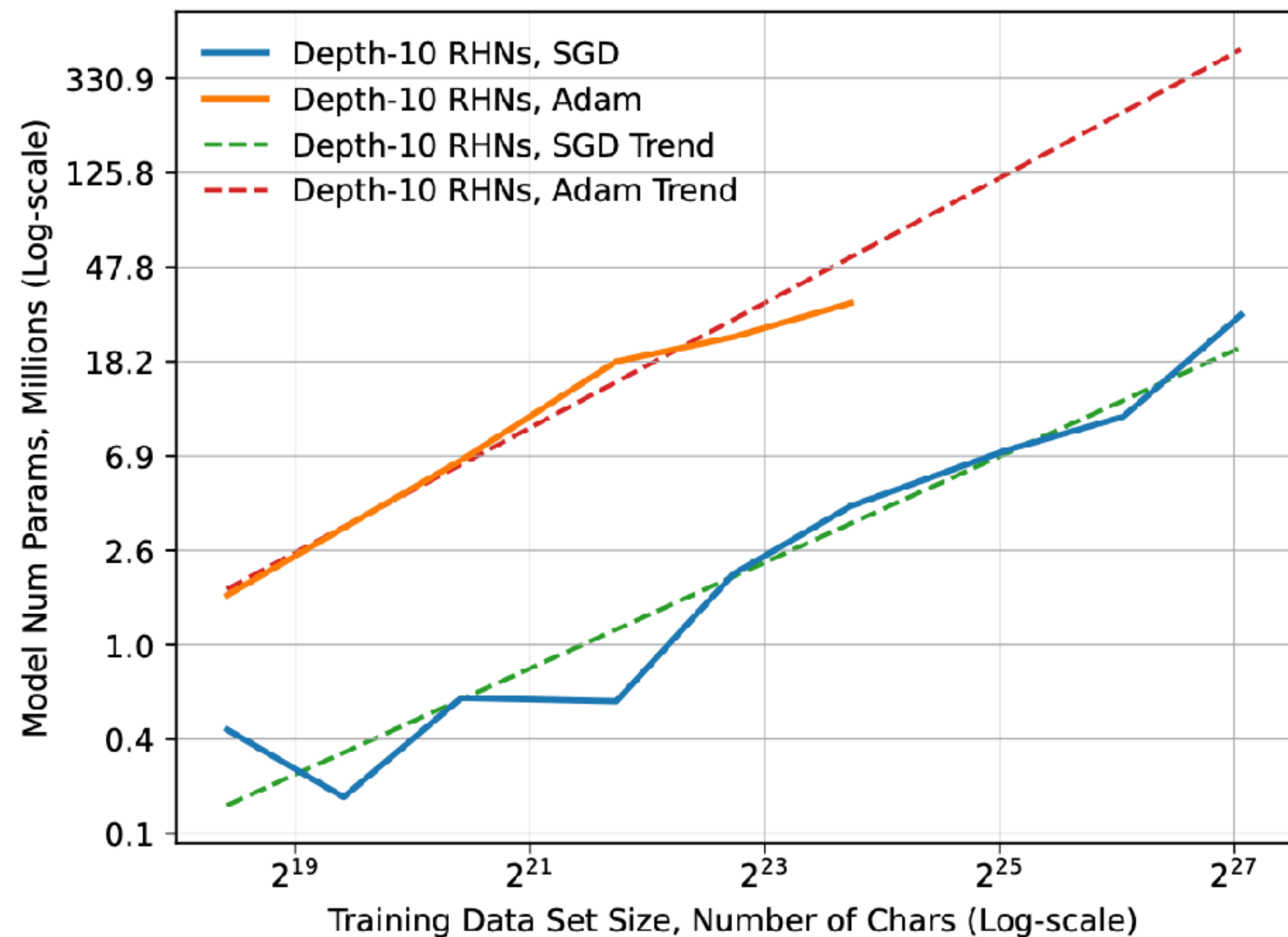
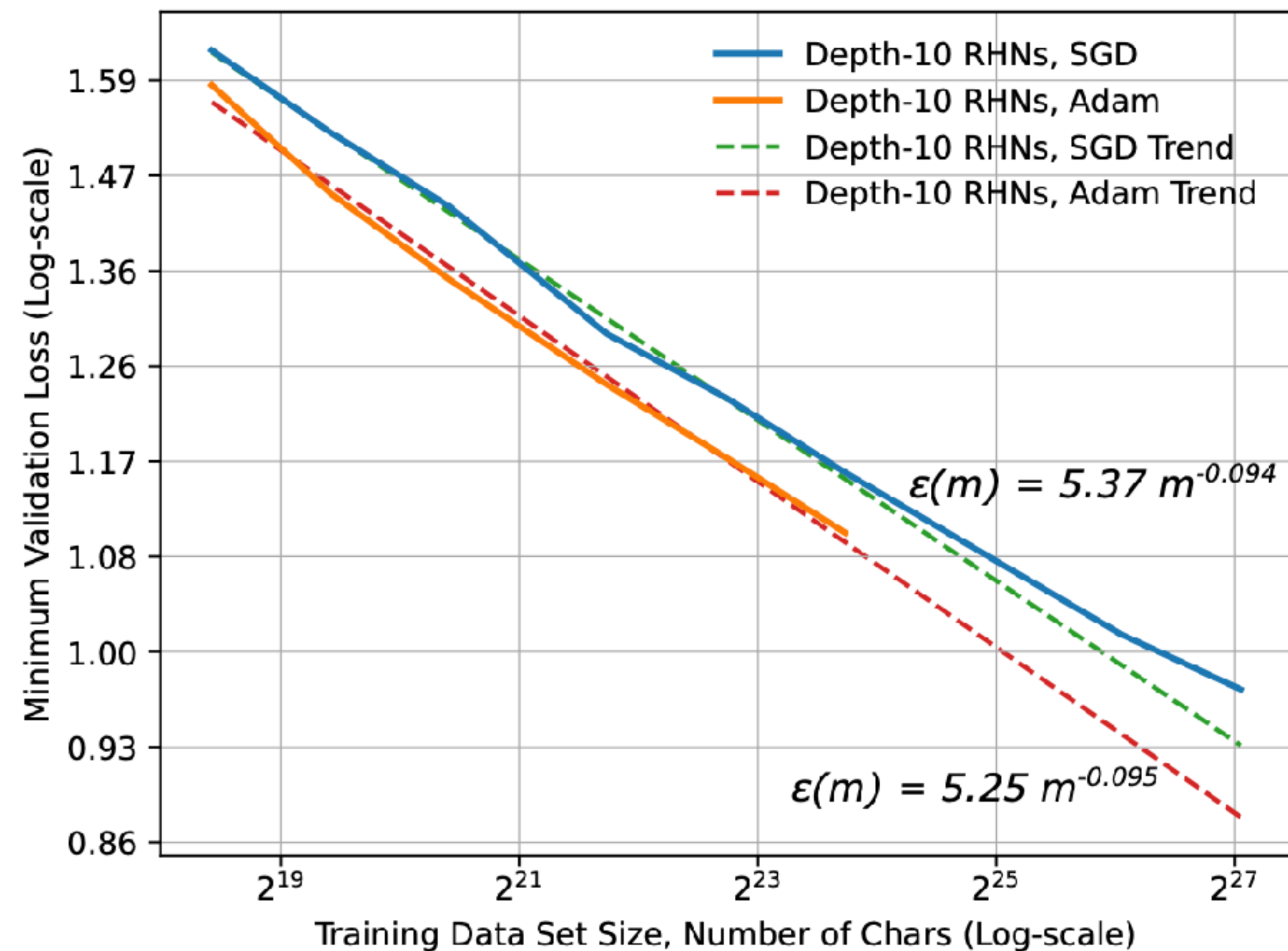


LSTM plateaus after <100 tokens
Transformer improves through the whole context



Scaling Laws

Optimizer



Scaling Laws

Maybe we can predict the impact of these decisions on big LMs before training!

The scaling law-based procedure:

1. Train a few smaller models with different settings
2. Establish a scaling law
3. Select the optimal hyperparameter configuration based on the scaling law prediction

Why do scaling laws appear?

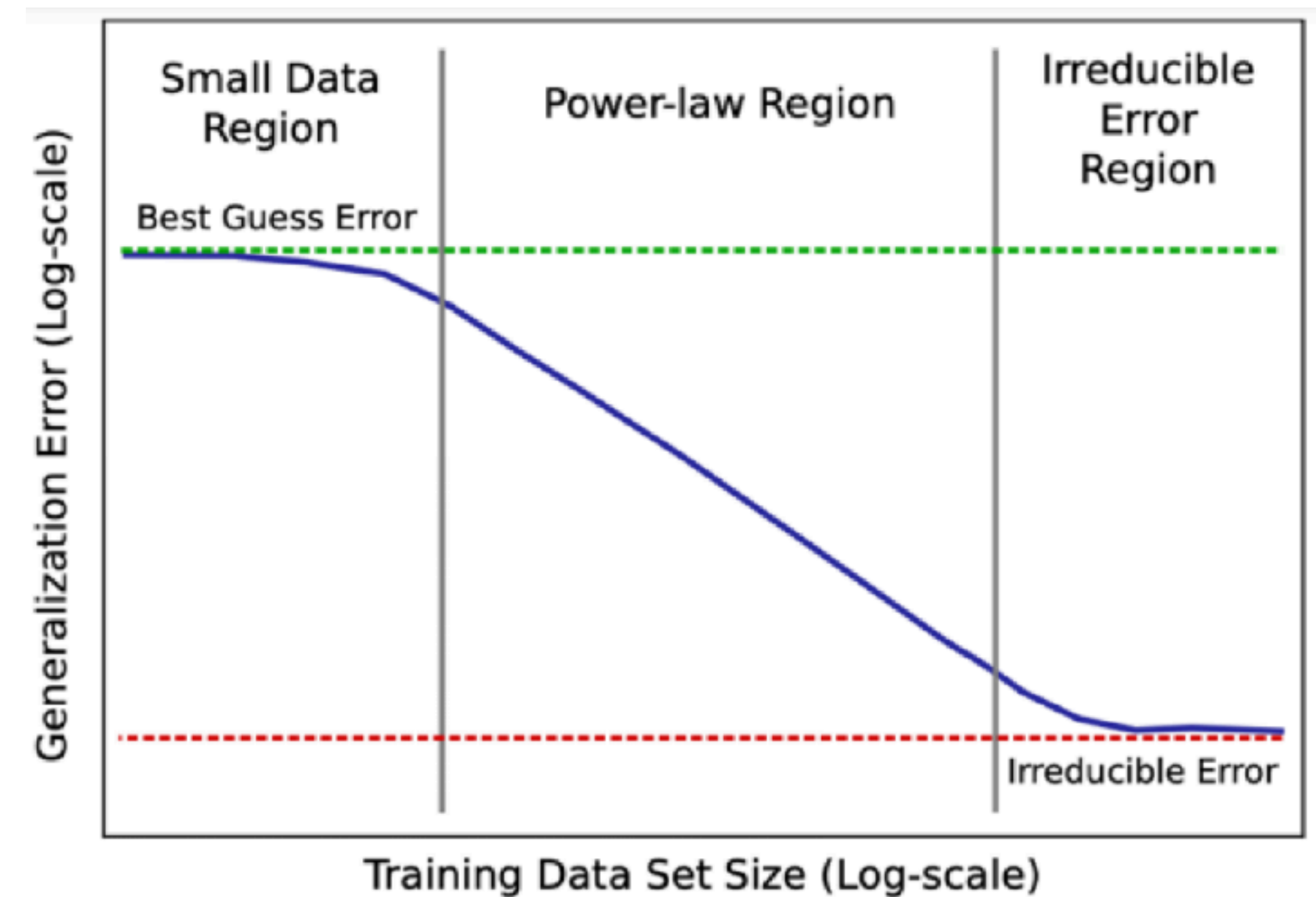
Classic ML theory predicts that error should reduce monotonically w.r.t. dataset size

Why is it a *power law*?

Key point: The loss naturally decays polynomially.

- *Why?*

[Hestness et al., 2017]



Example

Let's say we're training an ML system to predict the mean of a dataset.

Given input dataset $x_1, \dots, x_n \sim N(\mu, \sigma^2)$, estimate $\hat{\mu}$ as $\frac{\sum_i x_i}{n}$

What will the error of our model be?

$$\mathbb{E} \left[(\hat{\mu} - \mu)^2 \right] = \frac{\sigma^2}{n}$$

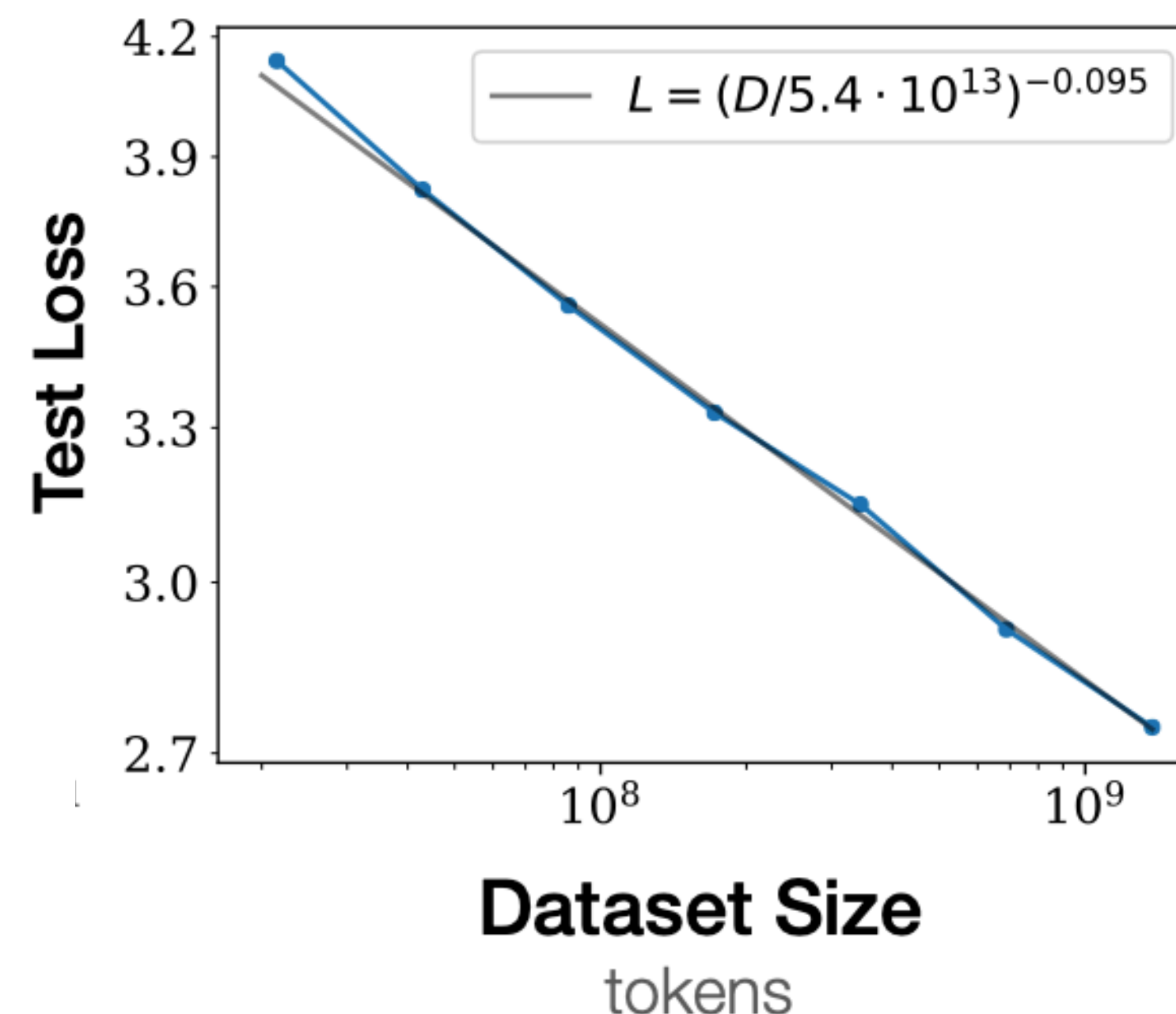
Surprise! This is a scaling law:

$$\log(\text{error}) = -\log n + 2 \log \sigma$$

Broadly speaking, any such polynomial decay $\frac{1}{n^\alpha}$ is a scaling law.

Scaling Laws for LLMs

- Most classical ML theory shows that models have $\frac{1}{n}$ scaling (note that $\alpha = 1$).
- This means we *should* see $\log y = -\log |D| + C$. But in practice...



- Why do neural models behave differently than the predictions of classic theory?

Neural Network Scaling Laws

Neural networks are powerful compared to many classical models. For example:

Given input dataset $x_1, \dots, x_n$ uniformly sampled from a 2D unit box $y_i = f(x_i) + N(0,1)$, estimate $f(x)$ by averaging over samples in the box.

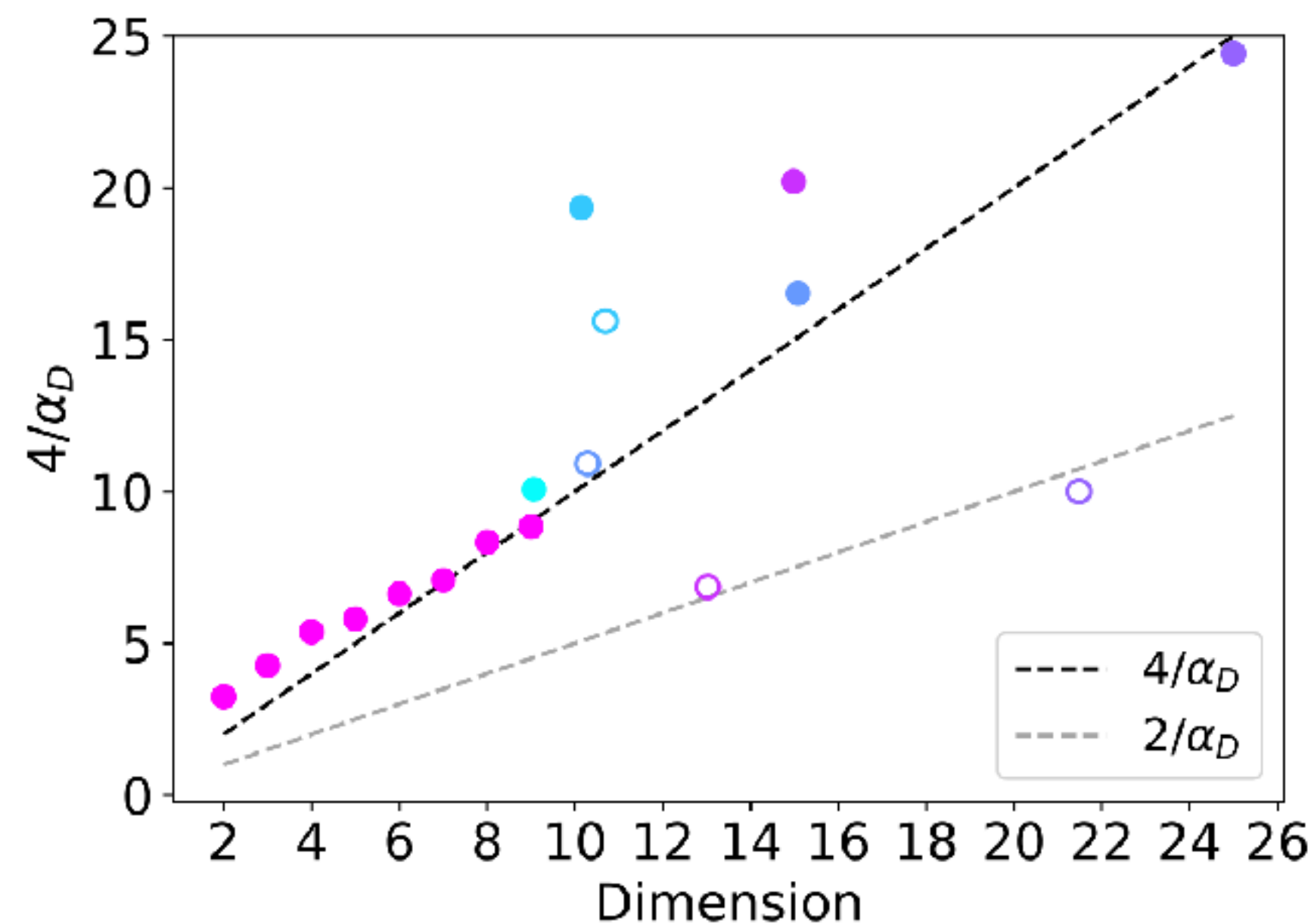
Let's slice the 2D space into boxes of length $n^{-1/4}$. We have roughly $\sqrt{n}$ boxes, and each box yields $\sqrt{n}$ samples. Thus, the error roughly scales with $\approx \frac{1}{\sqrt{n}}$.

In d dimensions, this scales as $n^{-1/d}$. We thus scale as $y = -\frac{1}{d}x + C$.

In other words, given powerful learning algorithms, we have **dimensionality-dependent scaling laws**.

Intrinsic Dimensionality

Some claim that scaling laws arise from polynomial rates of learning $\frac{k}{n^\alpha}$, where α depends on the intrinsic dimensionality of the task.



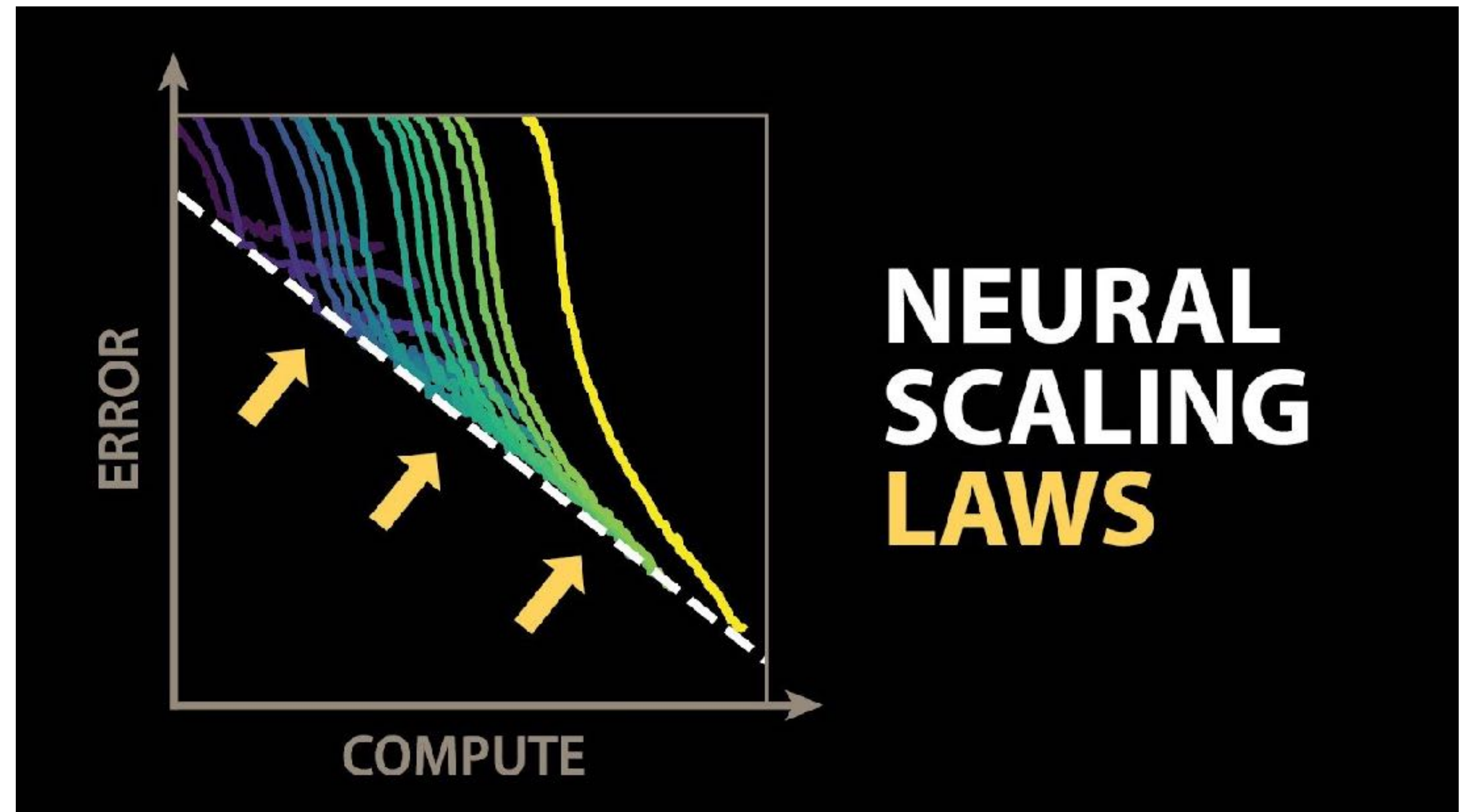
The pink points are generated by a teacher model, and have known intrinsic dimensionality.

Learning on real data aligns decently, but not perfectly. (Estimating intrinsic dimensionality is hard.)

Intrinsic Dimensionality

- There's a great video about this by Welch Labs specifically about language models:

- Highly recommend!

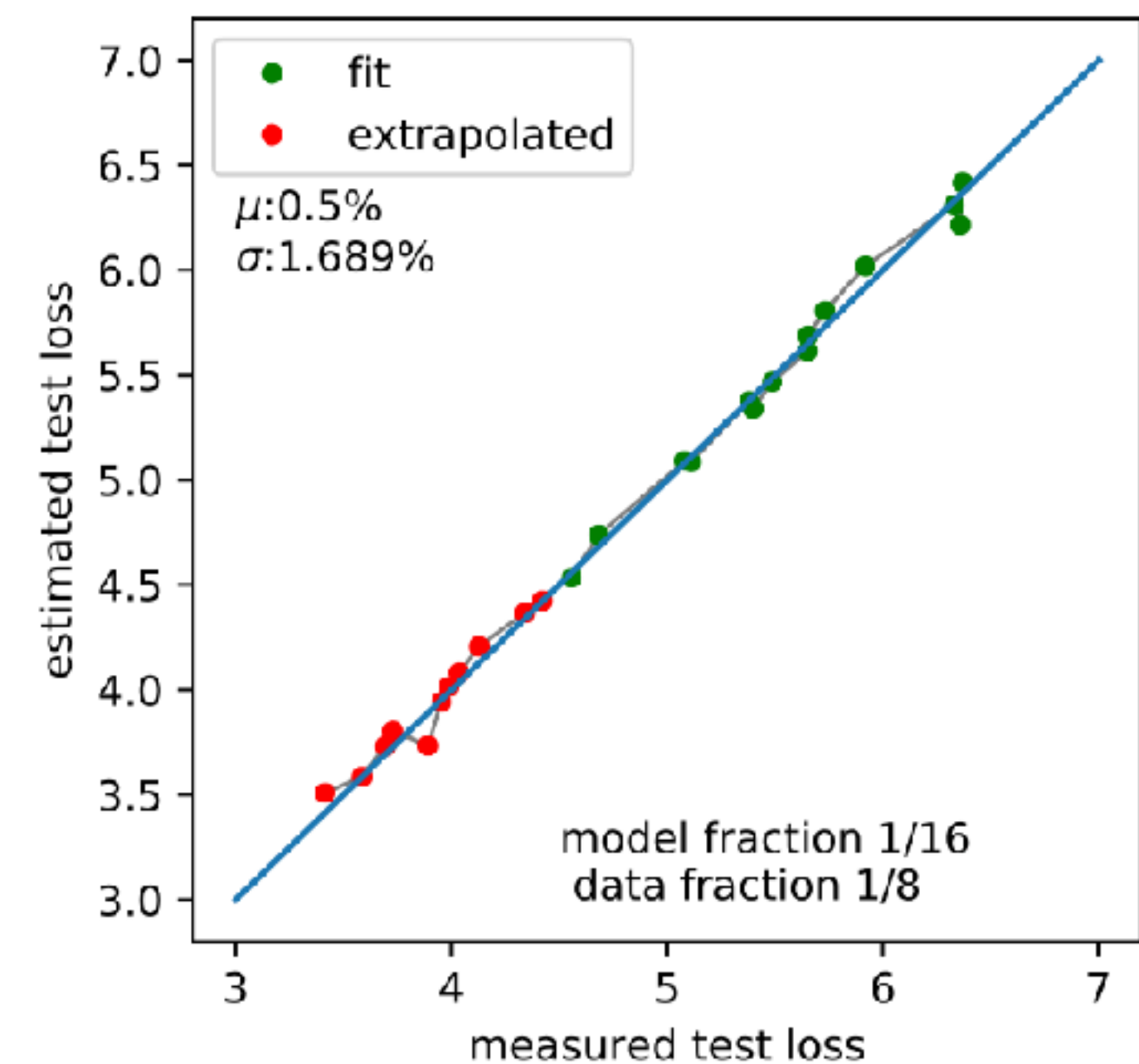
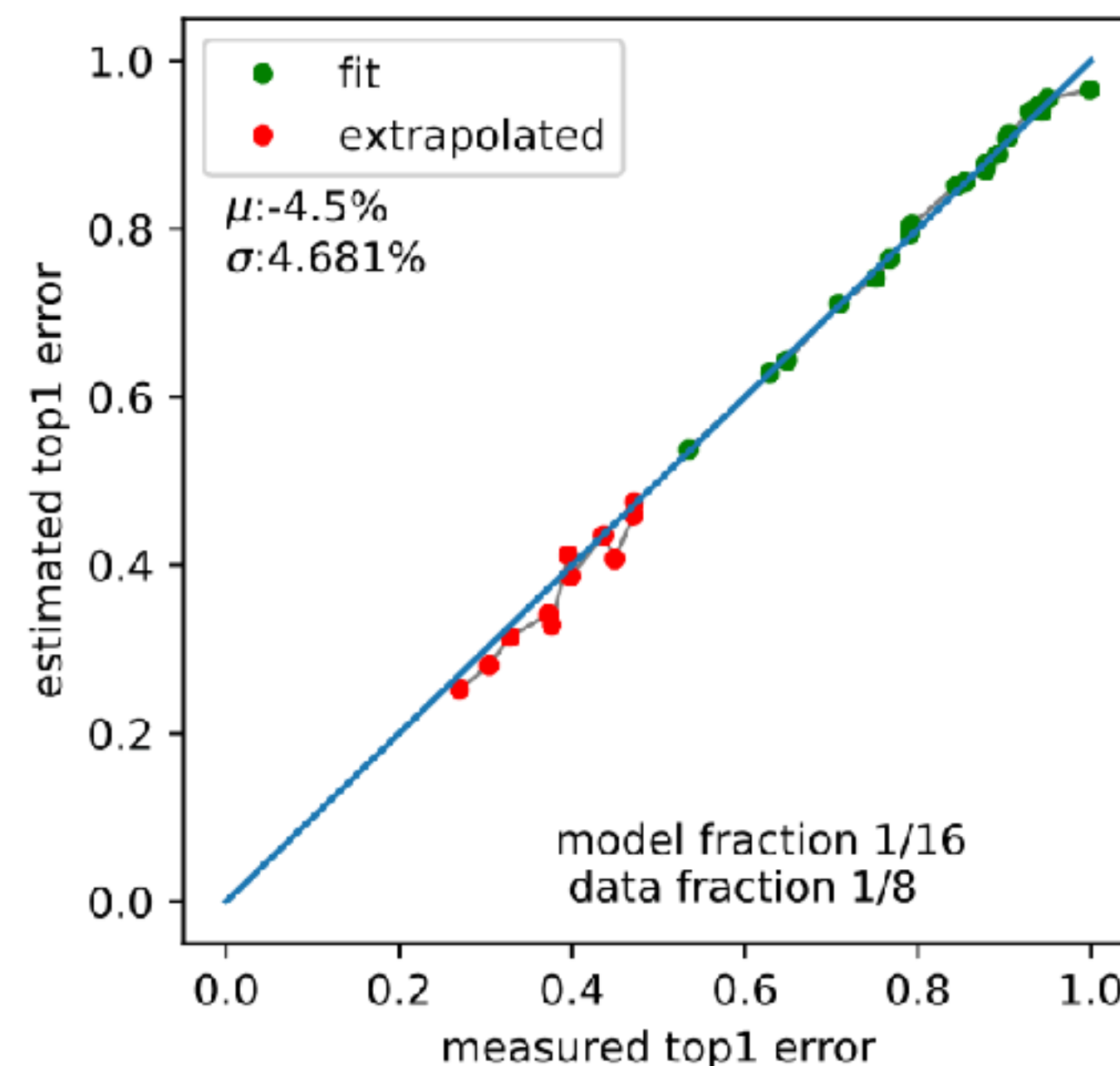


Predicting Optimal Model Size for Our Data

- **Joint data-model scaling laws** describe how the number of parameters relates to the size of the dataset w.r.t. reducing the loss:

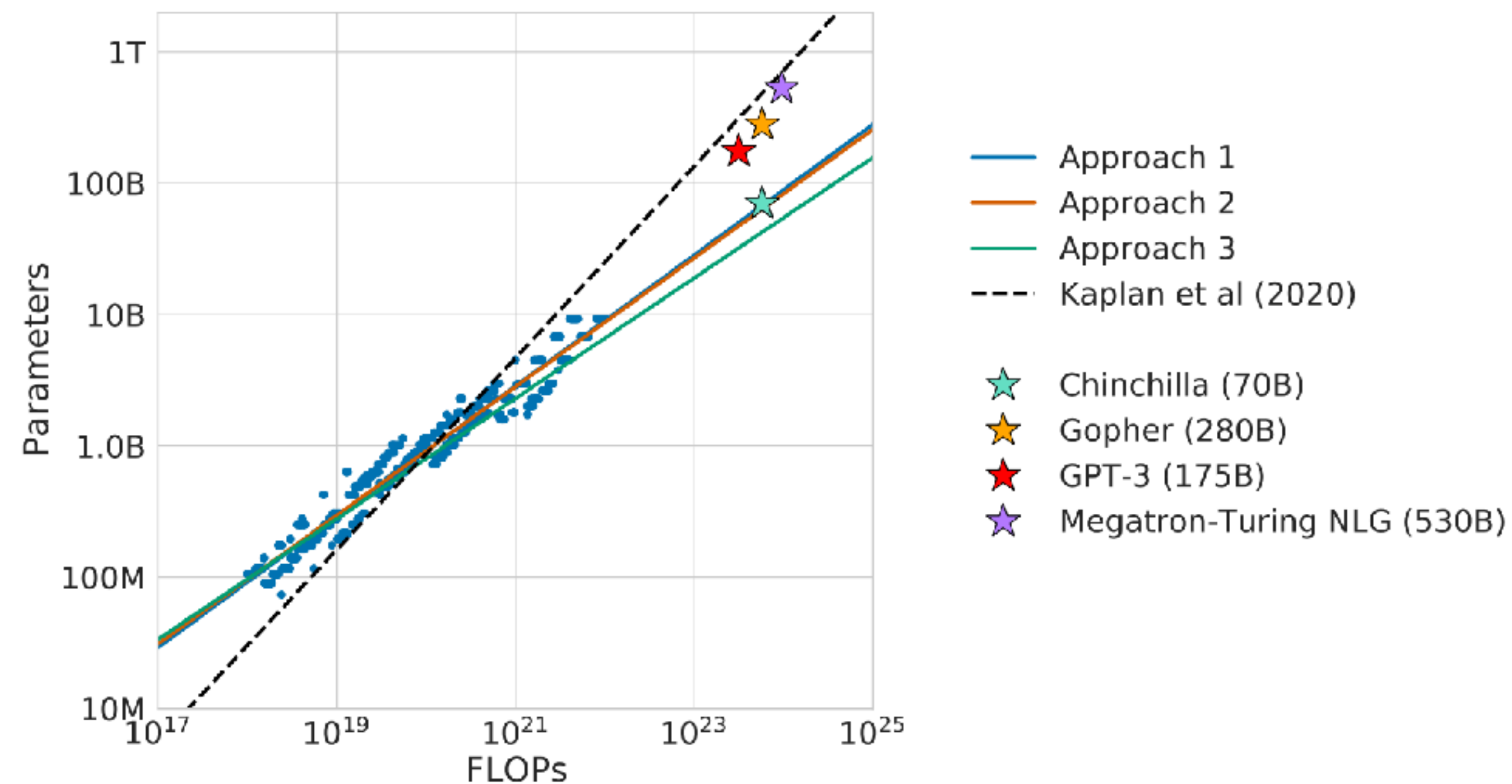
$$\text{Error} = (m^{-\alpha} + n^{-1})^{\beta}$$

- **Rosenfeld [2020]** fits the exponents on small models and small data, and then predicts scaling to larger values.



Optimizing w.r.t. Compute

- Our predictions so far only take data and model size into account.
- **Hoffmann et al. [2022]** argue that these predictions are pretty off-base:



It turns out that we need to use more data with smaller models.

They suggest 3 better methods for fitting scaling laws. Let's go over one of them.

Optimizing w.r.t. Compute

Pick a range of FLOPs budgets, vary parameter count, take the minimum over these convex regions that emerge to fit the scaling law.

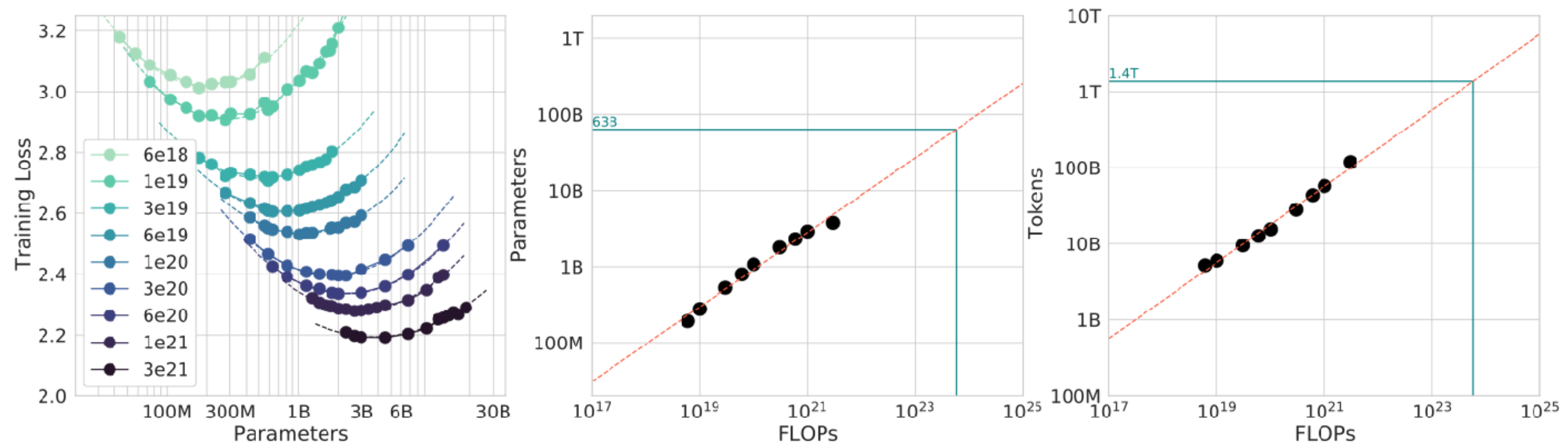


Figure 3 | **IsoFLOP curves.** For various model sizes, we choose the number of training tokens such that the final FLOPs is a constant. The cosine cycle length is set to match the target FLOP count. We find a clear valley in loss, meaning that for a given FLOP budget there is an optimal model to train (**left**). Using the location of these valleys, we project optimal model size and number of tokens for larger models (**center** and **right**). In green, we show the estimated number of parameters and tokens for an *optimal* model trained with the compute budget of *Gopher*.

Deployment Considerations

- It's well and good to optimize models given a certain compute budget.
 - In practice, though, most of the compute is *inference* once you've deployed your model.
- The more usage you expect, the more it becomes worth it to pay the upfront cost of "overtraining" - training on more data than is compute-optimal

Data-constrained Scaling Laws

- So far, we've assumed that we have infinite data and never complete 1 epoch
- How do repeated examples affect scaling?

Effective data

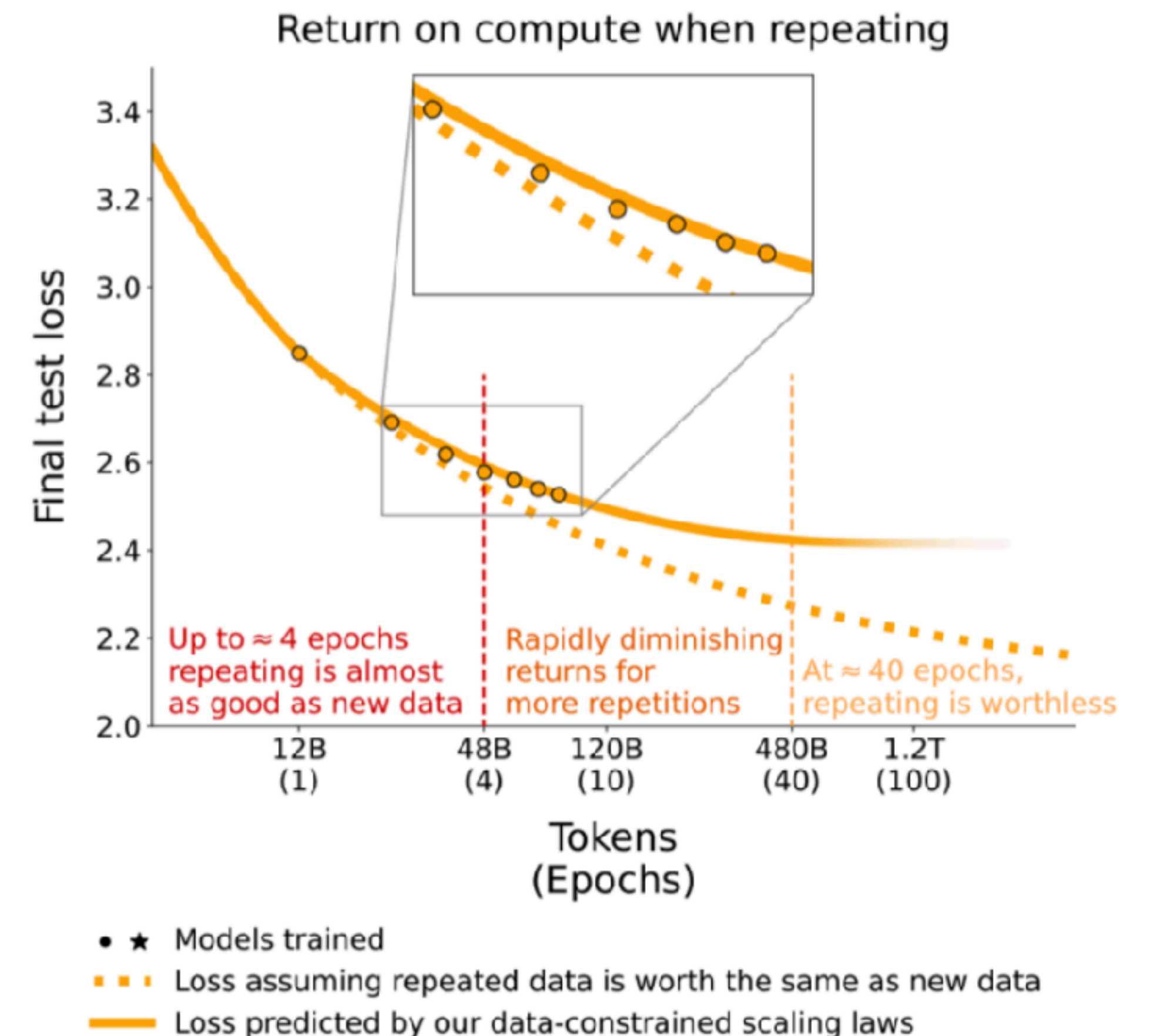
$$D' = U_D + U_D R_D^* (1 - e^{-\frac{R_D}{R_D^*}})$$

Unique tokens

Constant

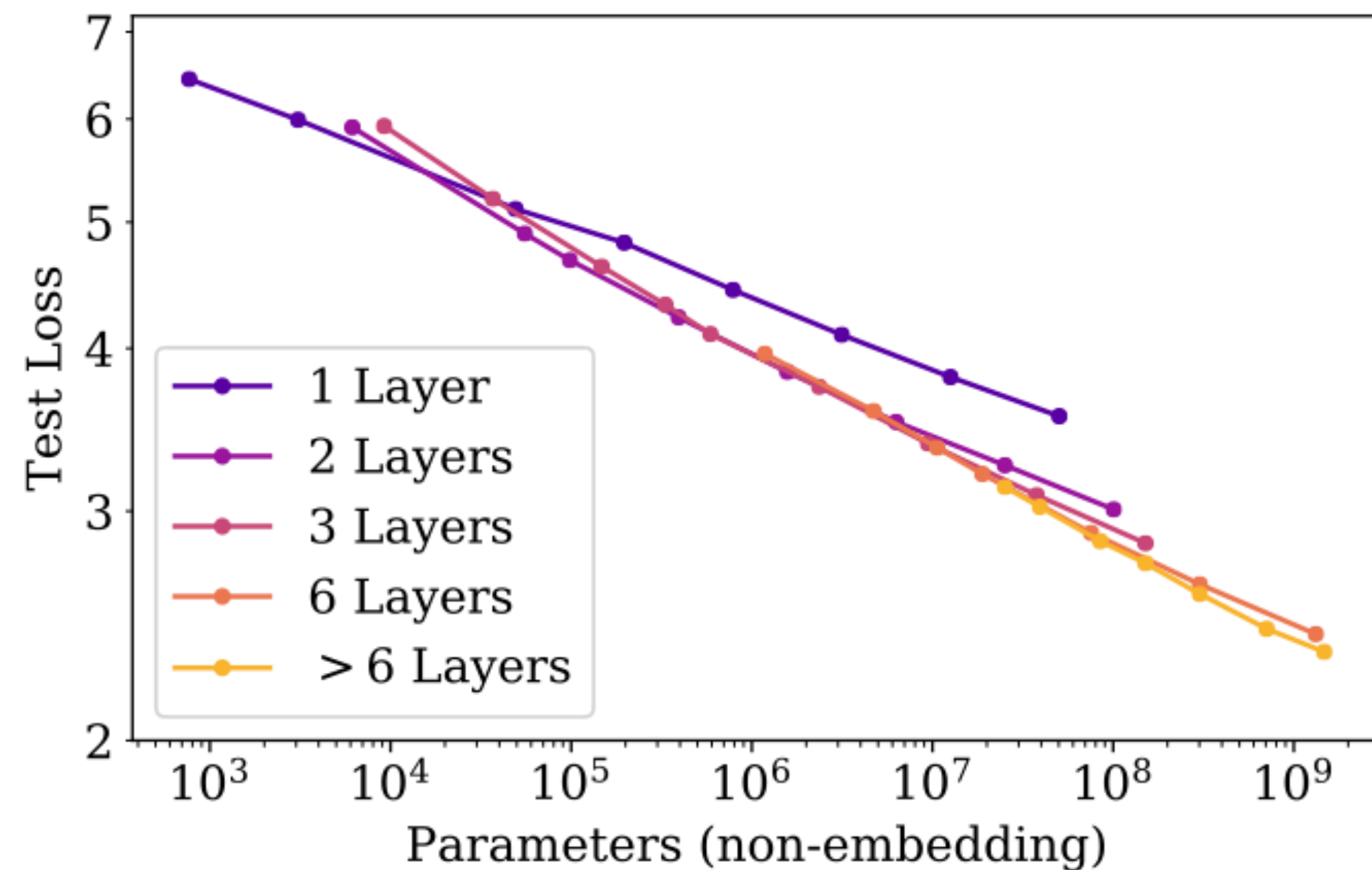
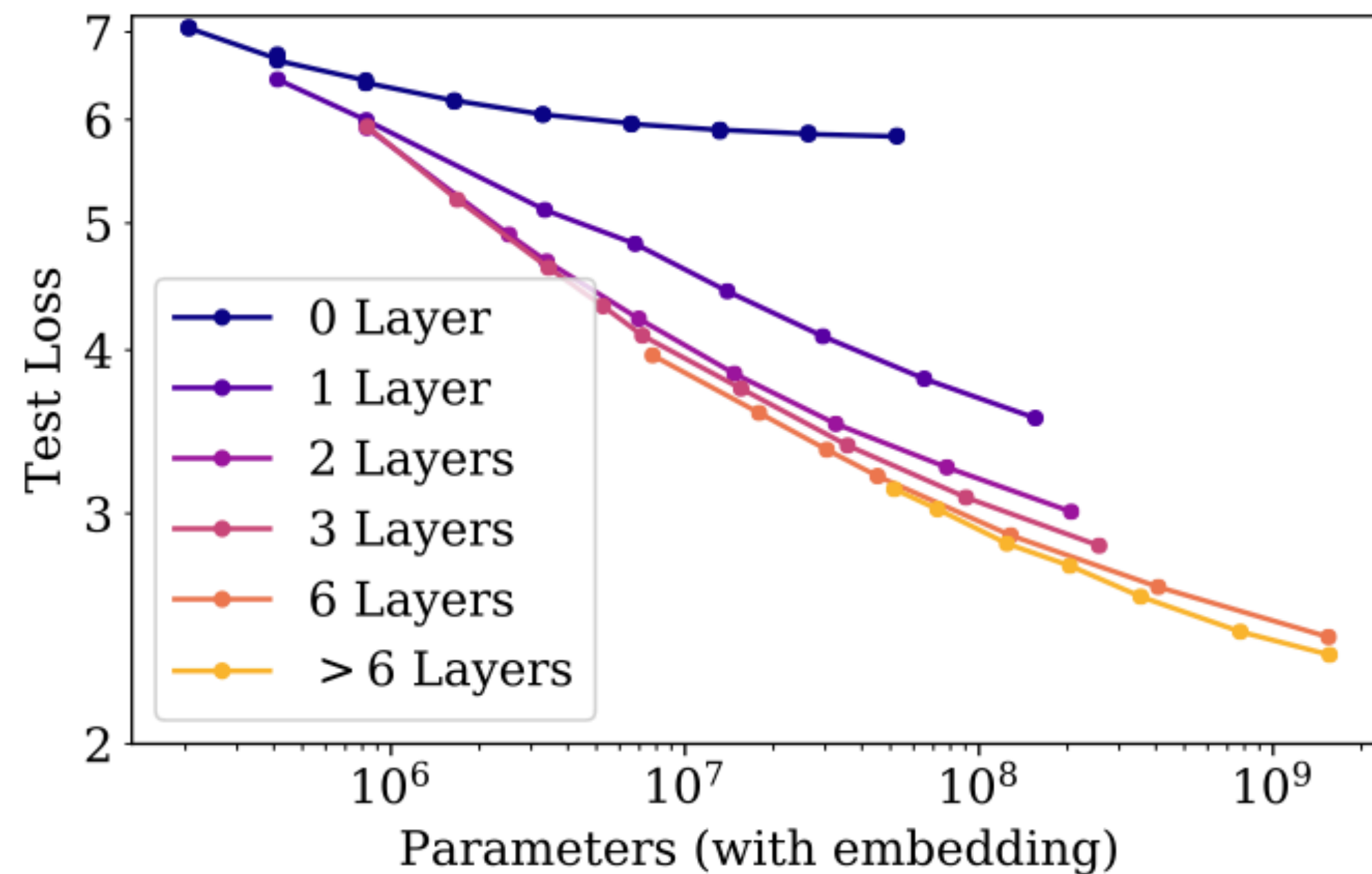
Repetition

[Muenninghoff et al., 2025]



Depth vs. Width

- Not all parameters are equal:



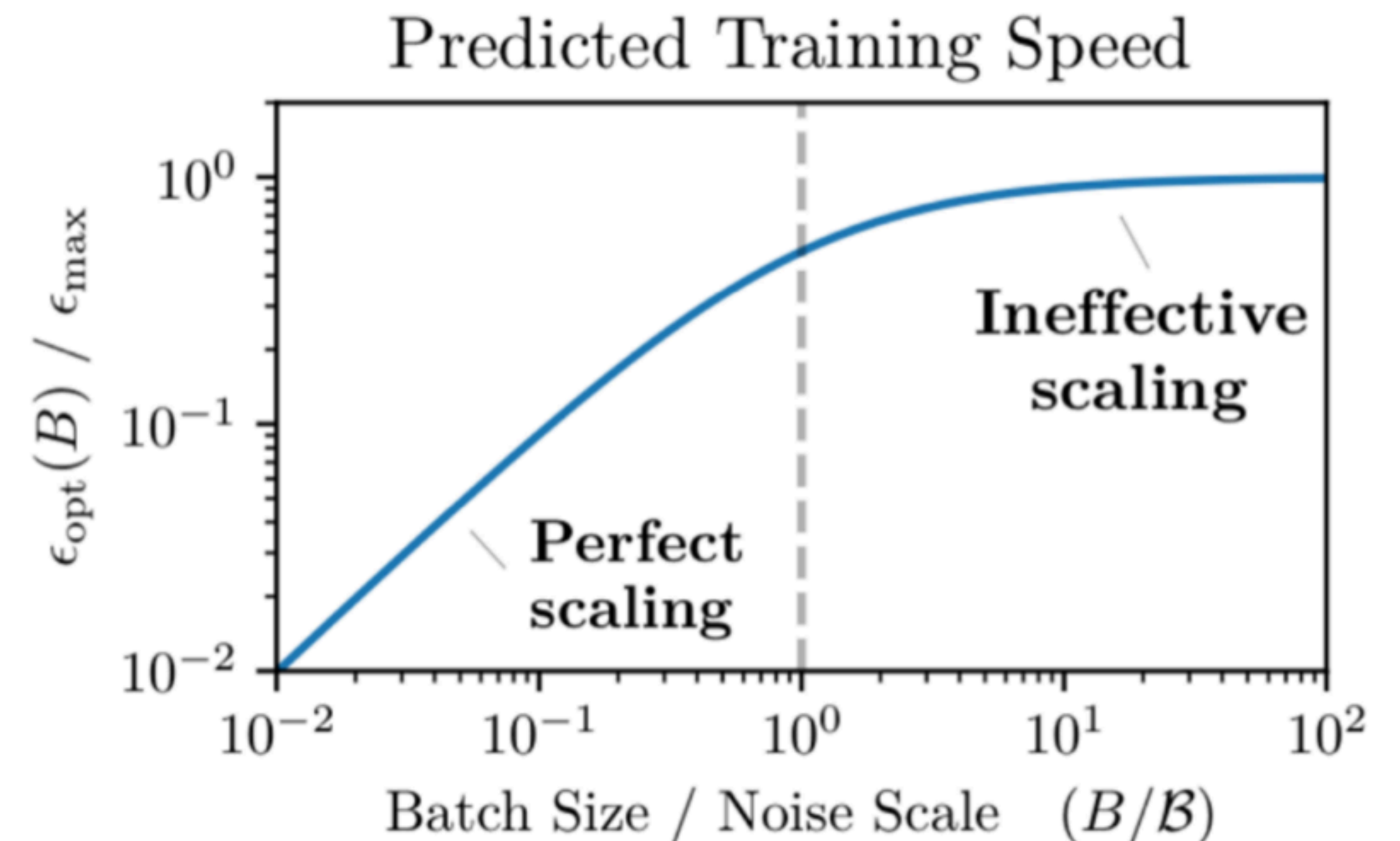
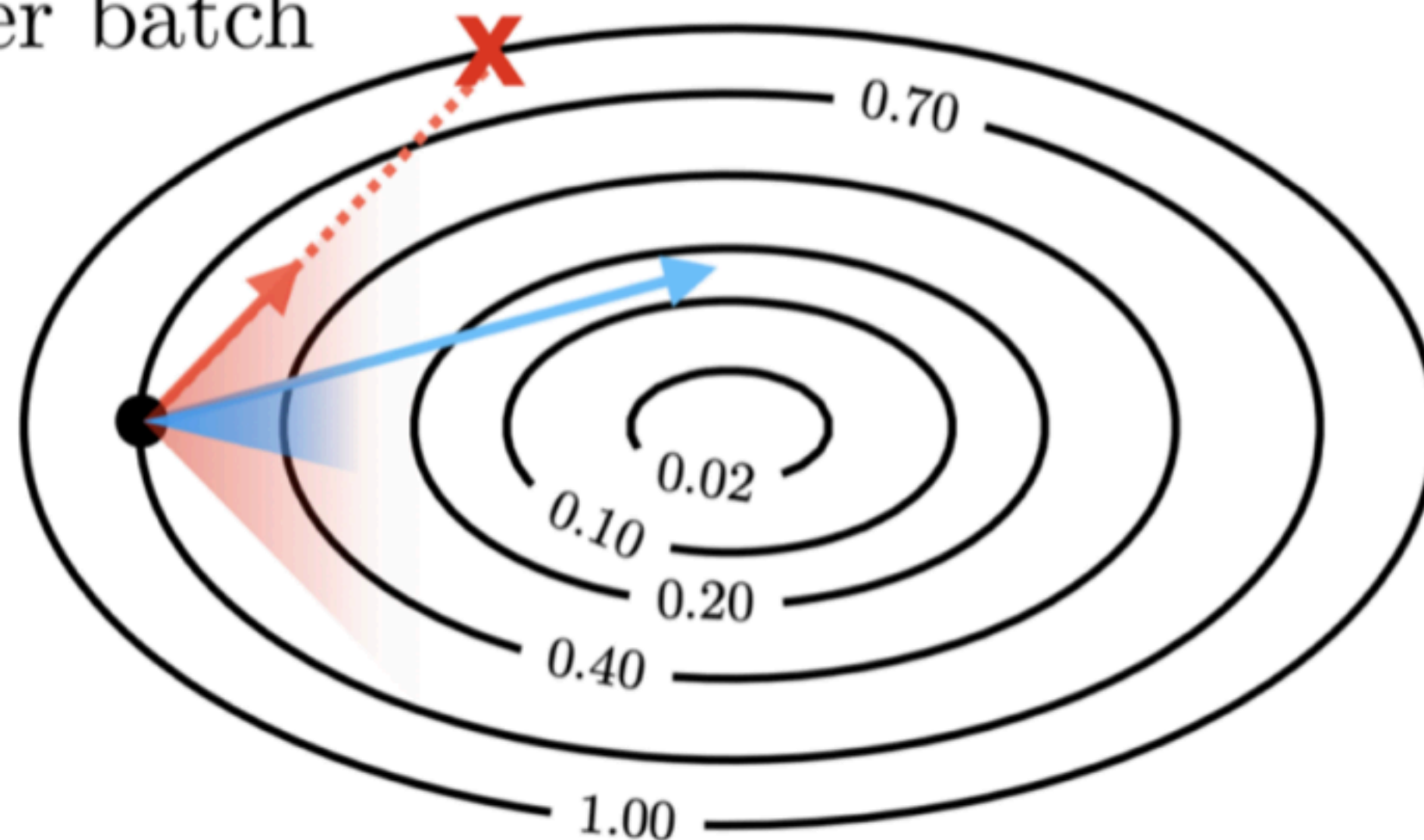
- Given a parameter budget, it's better to make the model "deeper" (add more layers) than "wider" (e.g., increase hidden dim) up to a couple layers. (Diminishing returns beyond this.)

Batch Size

- Known to have quickly diminishing returns past a certain point
- **Critical batch size:** the batch size that would provide a roughly optimal compromise between time and compute efficiency for a given target loss

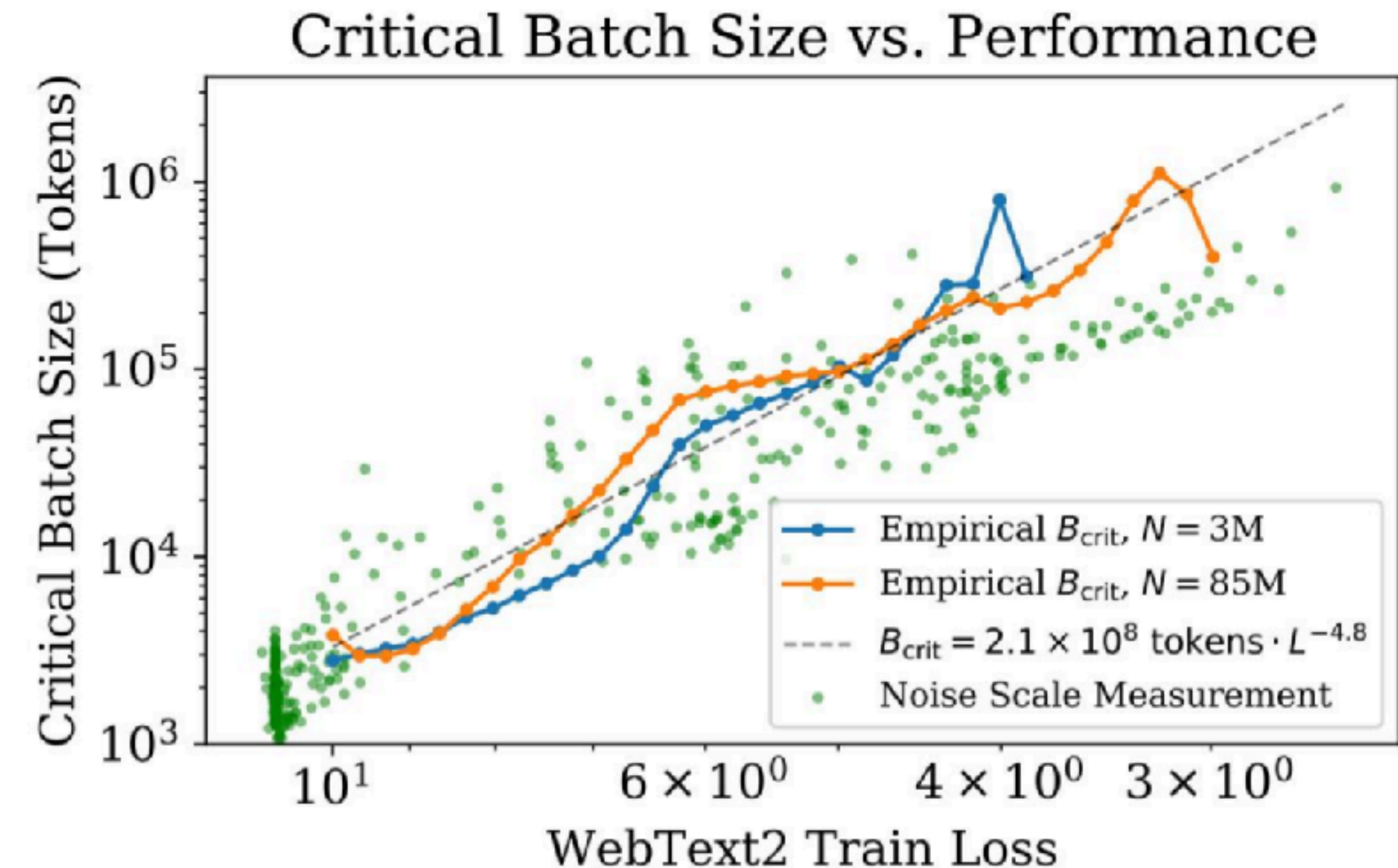
■ Smaller batch

■ Larger batch

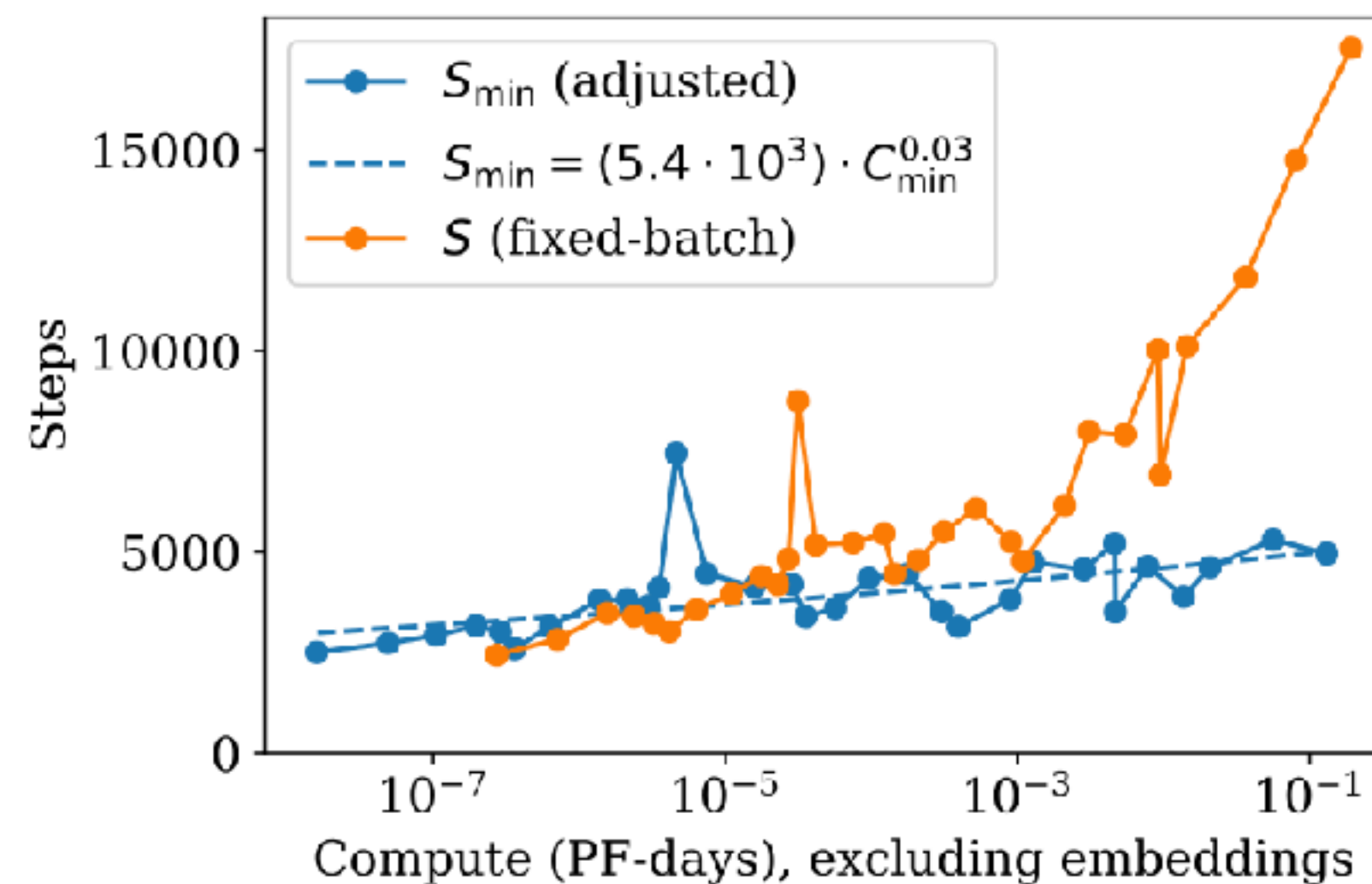


Batch Size

- The smaller the target loss, the larger the critical batch size (in tokens)



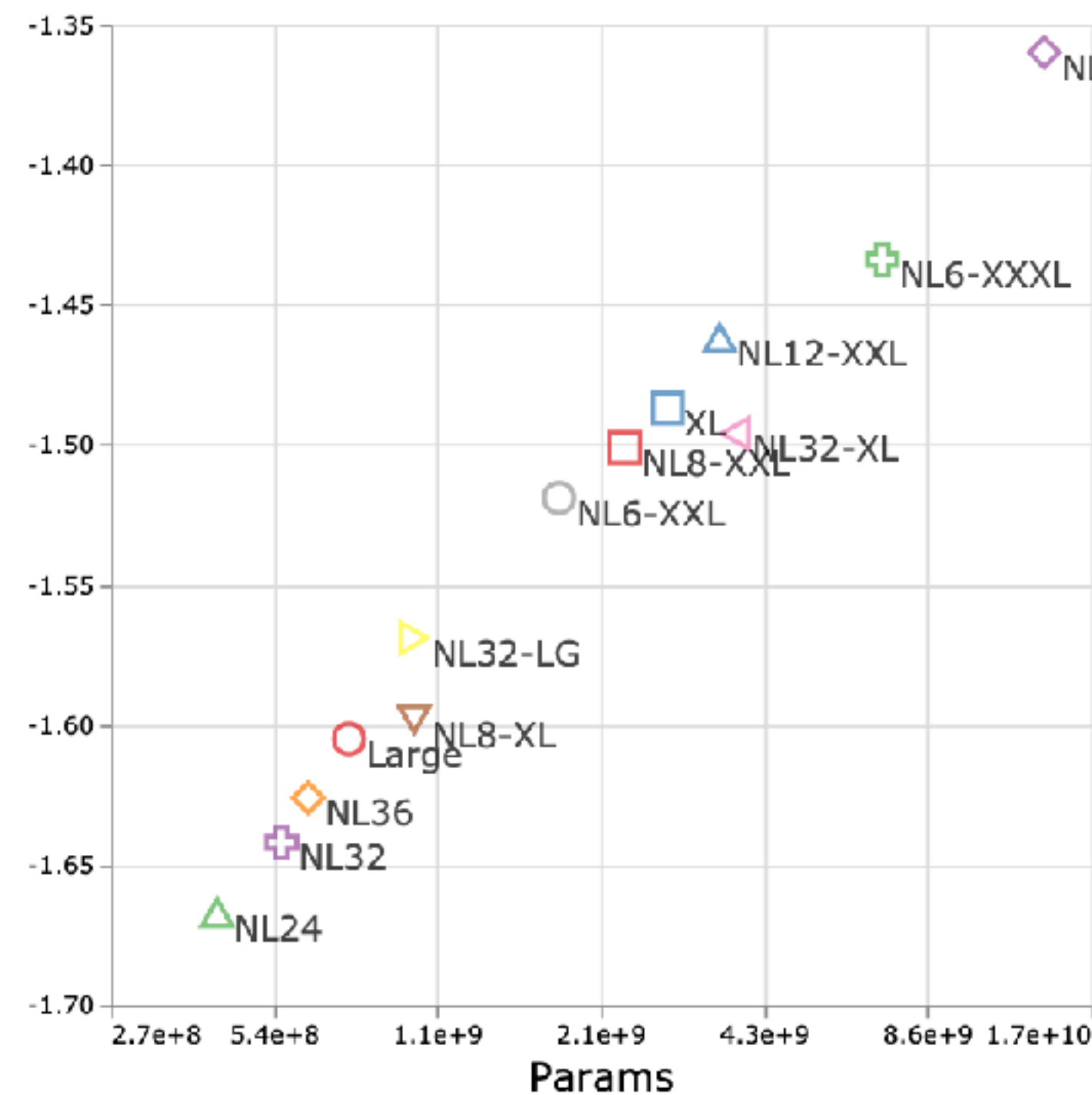
- As model/data size increases, it's better to increase batch sizes rather than the number of sequential training steps



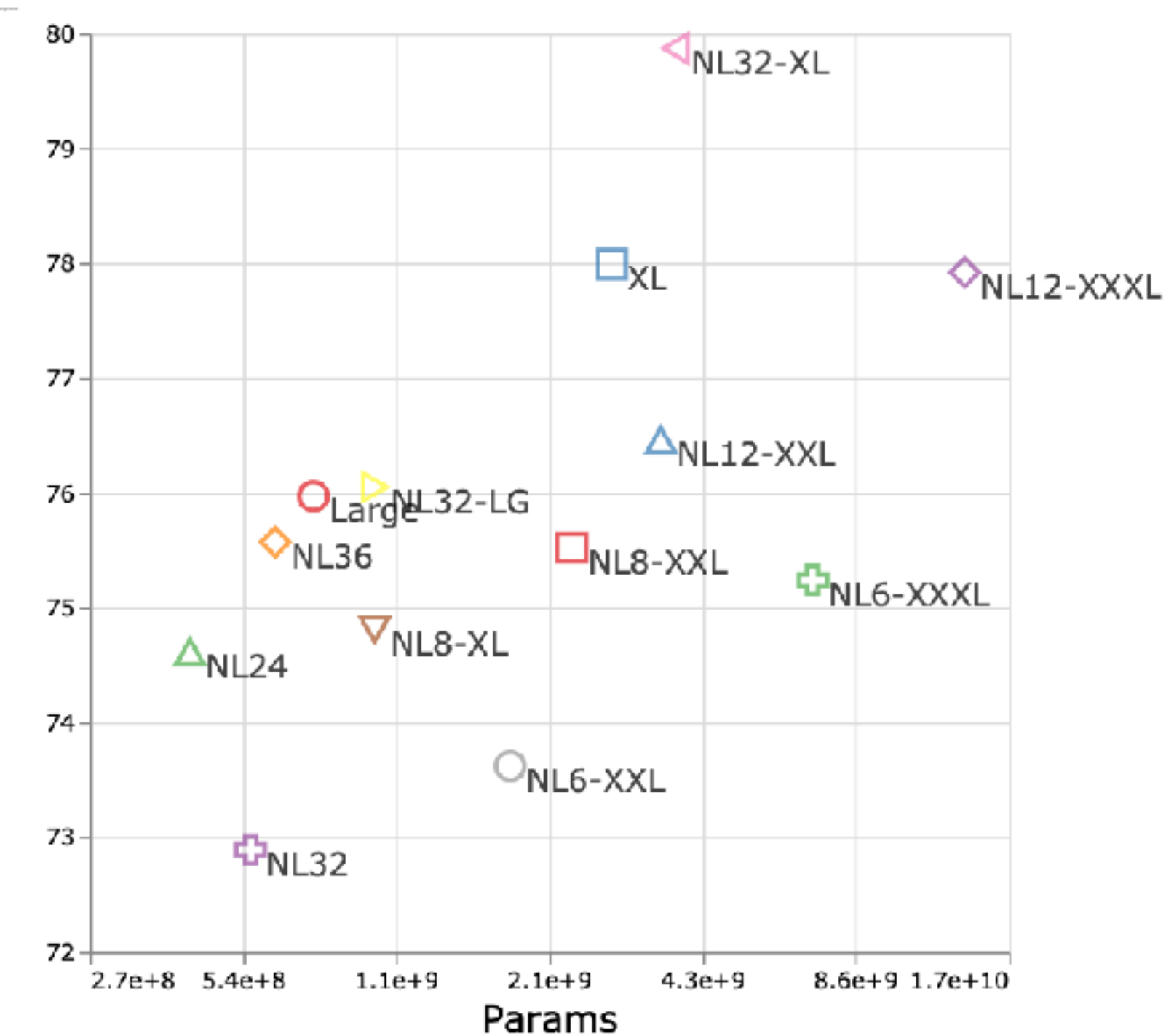
Implications for Downstream Evaluation

- Note that this is all about how **losses** scale w.r.t. compute
- The relationship between this and downstream task performance is quite messy

- This will become relevant when we discuss emergence in a moment



(a) Pre-training scaling



(b) Fine-tuning scaling

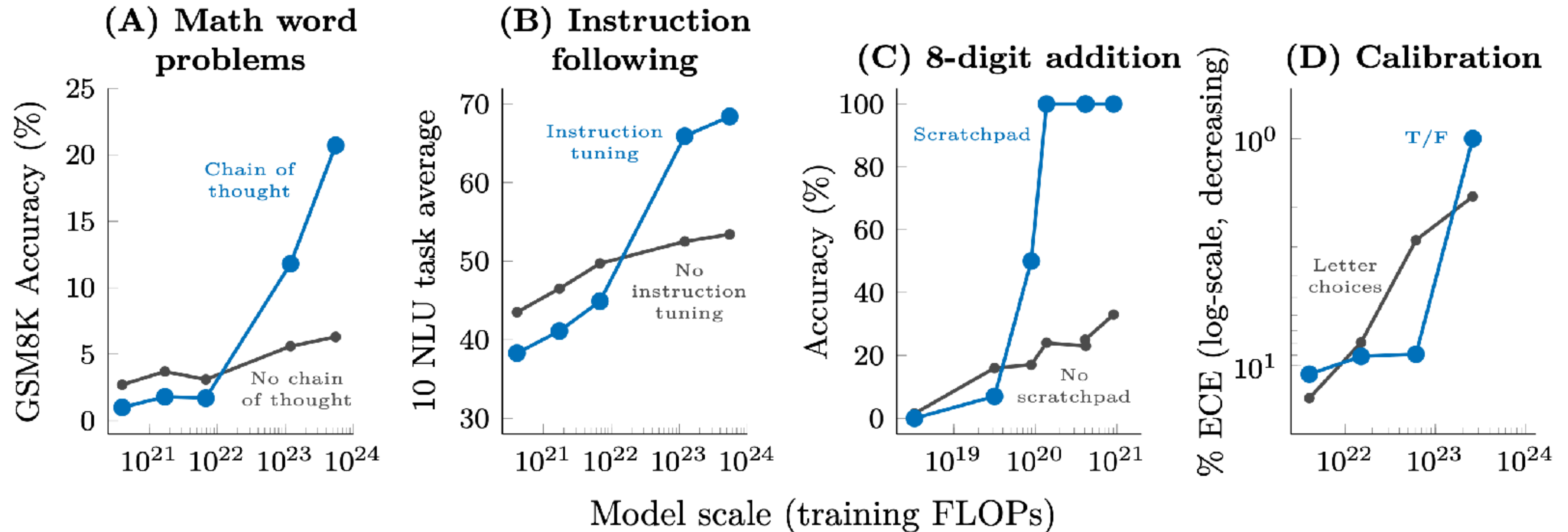
Emergence and In-context Learning

GPT-3 and Beyond

- Depending on who you ask, the era of “large” language models (LLMs) started with either:
 - 2018: GPT - first Transformer-based LM trained on a decently large corpus
 - 2020: GPT-3 - first general-purpose LM that achieved great performance on many tasks
- Once you reach a certain point, a model gains the hugely useful ability to do things like **in-context learning**.
 - That is, learning from just text without any gradient updates. (More on this later.)
- The first model that was consistently able to do in-context learning well was GPT-3.
 - 175 billion parameters!

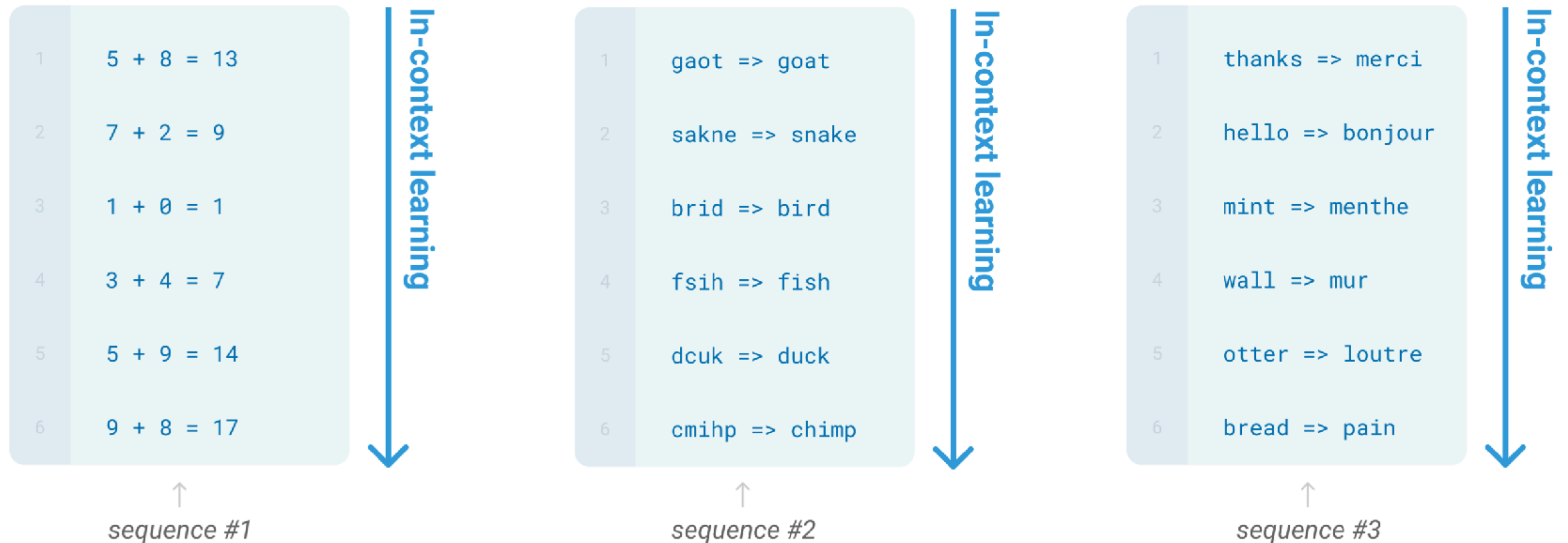
Emergence?

[Wei et al., 2022]



- Not all capabilities have smoothly increasing accuracies.
- When a capability arises suddenly/discontinuously, we call it **emergence**.

A Powerful “Emergent” Ability



This paradigm of giving a model examples of a task in-context and having it learn to do the task with no parameter updates is called **in-context learning**.

Prompting

- Starting with GPT-3 (2020), language models started being able to handle new tasks given only few-shot demonstrations in the prompt, **with no gradient updates**
 - It was hypothesized that this came from exposure during pre-training. E.g., maybe it's seen many examples of translation online

Three reasons to start a succulent garden

1. Succulents are low-maintenance: They don't require much watering or fertilizing, and they can tolerate a wide range of light conditions.
2. Succulents are drought-tolerant: They're perfect for areas that receive little rainfall or irrigation.
3. Succulents add interest and variety to the landscape: With their wide range of shapes, sizes, and colors, they can provide a unique and eye-catching addition to any garden.

Zero-shot Prompting

- Given a single unlabeled datapoint $\mathbf{x}$, we want to predict label y

$\mathbf{x}$ = "The other day, I saw an alright movie. Wasn't great, really."
Was the previous sentence positive or negative?

This part of the prompt can be tuned. It is often called a **template**.



The mapping of labels to tokens is called the **verbalizer**.

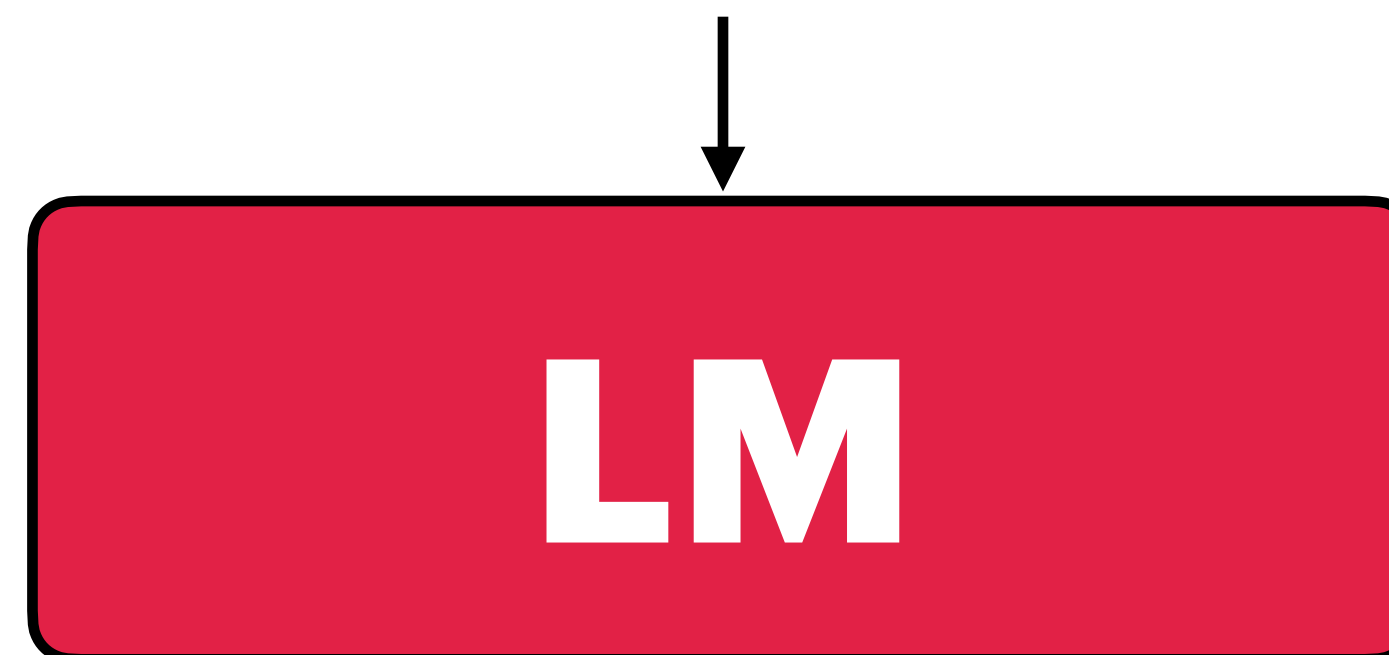
y = *negative*

Zero-shot Prompting

- Given a single unlabeled datapoint $\mathbf{x}$, we want to predict label y

$\mathbf{x}$ = "The other day, I saw an alright movie. Wasn't great, really."
If I told you this, would you want to see the movie? Say yes or no.

This part of the prompt
can be tuned. It is often
called a **template**.



The mapping of labels to
tokens is called the **verbalizer**.

$y =$ no

Zero-shot Prompting

- Given a single unlabeled datapoint $\mathbf{x}$, we want to predict label y

$\mathbf{x} = \text{"\{example\}"}$

If I told you this, would you want to see the movie? Say yes or no.

This part of the prompt can be tuned. It is often called a **template**.



The mapping of labels to tokens is called the **verbalizer**.

$y = no$

Few-shot Prompting

Given a few *labeled* datapoints $(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots$, called **demonstrations**, and an unlabeled datapoint $\mathbf{x}_n$, we want to predict label y_n .

The following are multiple choice questions about high school computer science.

Let $x = 1$. What is $x \ll 3$ in Python 3?

(A) 1 (B) 3 (C) 8 (D) 16

Answer: C

Which is the largest asymptotically?

(A) $O(1)$ (B) $O(n)$ (C) $O(n^2)$ (D) $O(\log(n))$

Answer: C

What is the output of the statement "a" + "ab" in Python 3?

(A) Error (B) aab (C) ab (D) a ab

Answer:

LM

B

Few-shot Prompting

Given a few *labeled* datapoints $(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots$, called **demonstrations**, and an unlabeled datapoint $\mathbf{x}_n$, we want to predict label y_n .

It is very easy to trick models, even accidentally! Notice that all the answers here are C, so the model might learn to just always answer "C".

The following are multiple choice questions about high school computer science.

Let $x = 1$. What is $x \ll 3$ in Python 3?

(A) 1 (B) 3 (C) 8 (D) 16

Answer: C

Which is the largest asymptotically?

(A) $O(1)$ (B) $O(n)$ (C) $O(n^2)$ (D) $O(\log(n))$

Answer: C

What is the output of the statement "a" + "ab" in Python 3?

(A) Error (B) aab (C) ab (D) a ab

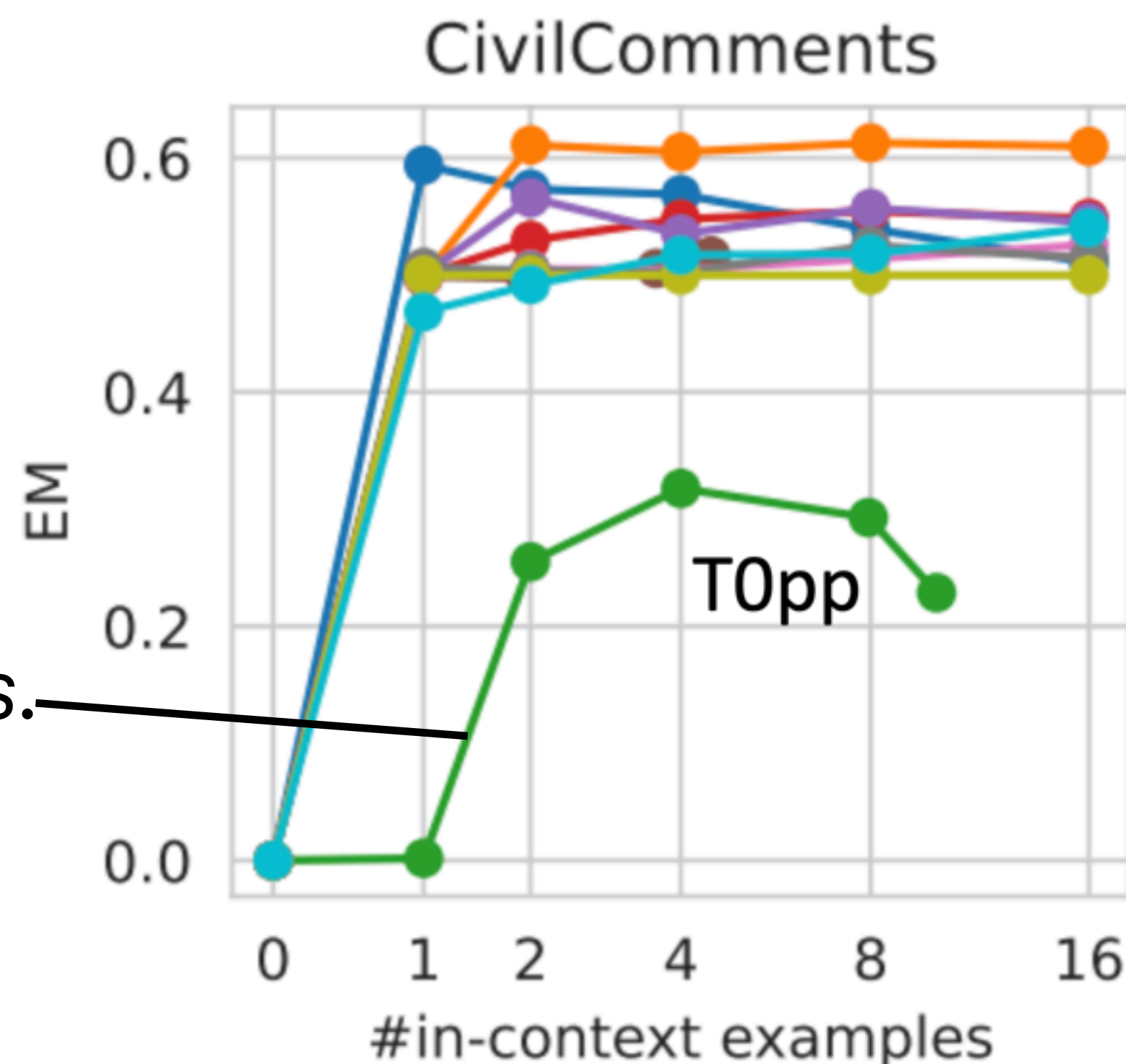
Answer:



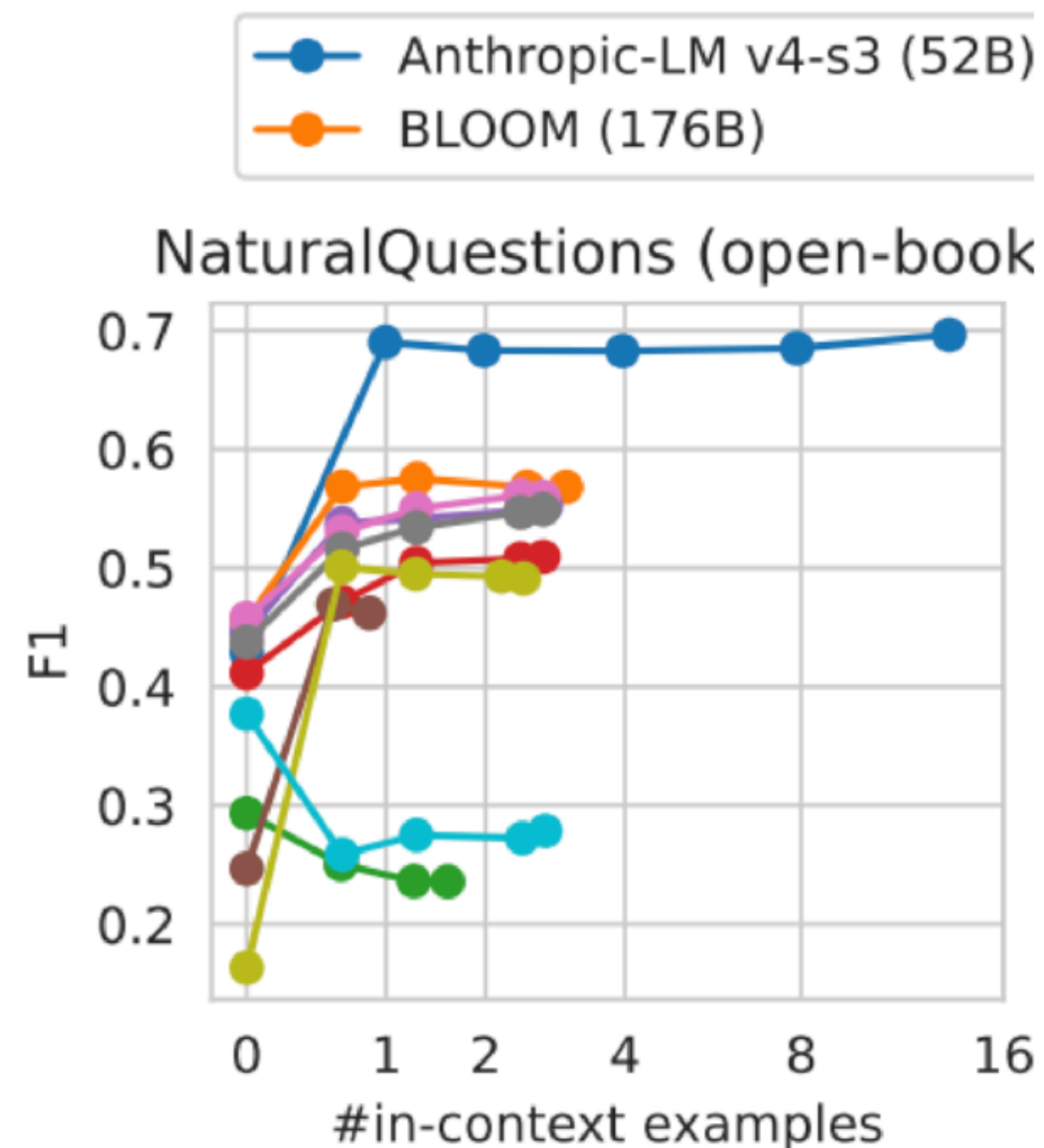
C

More demonstrations is (usually) better.

As we increase the number of in-context demonstrations, performance usually increases.



But not always.



[Liang et al., 2022]

A Sneak Peek

[Park et al., 2025]

(a) Words on a grid

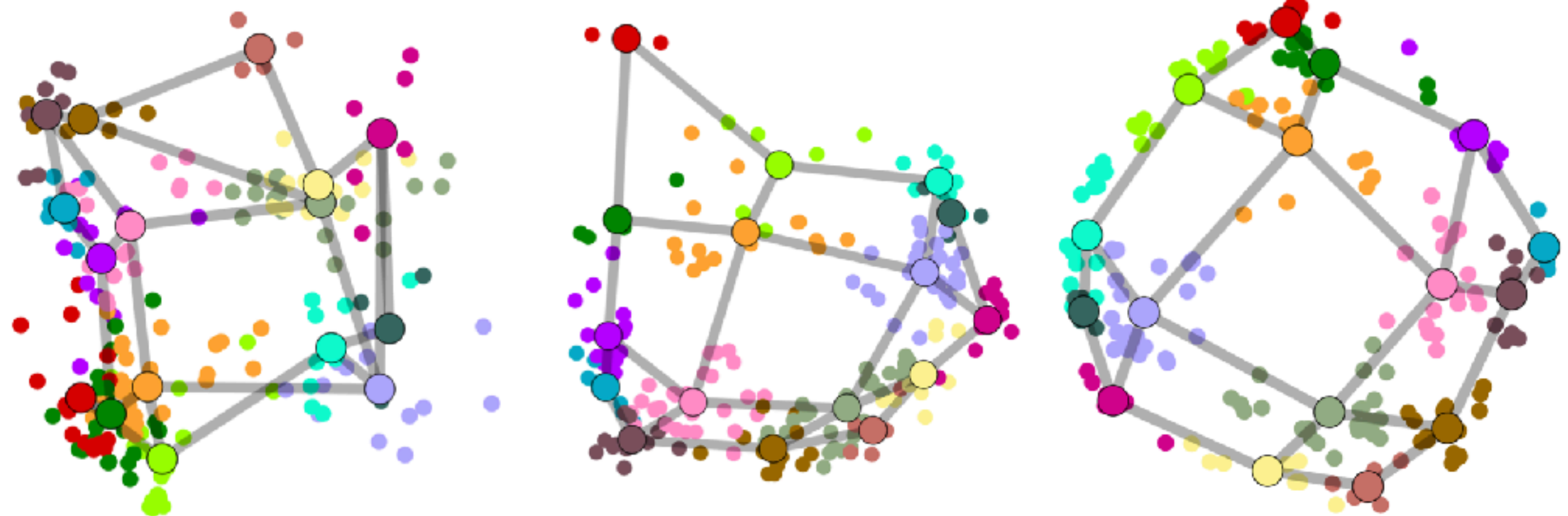
apple	bird	car	egg
house	milk	plane	opera
box	sand	sun	mango
rock	math	code	phone

(b) Data generation

Random walk on a grid:

"apple, bird, milk, sand, sun, plane, opera, ..."

(c) Emergent grid representation in context



Context length: 200

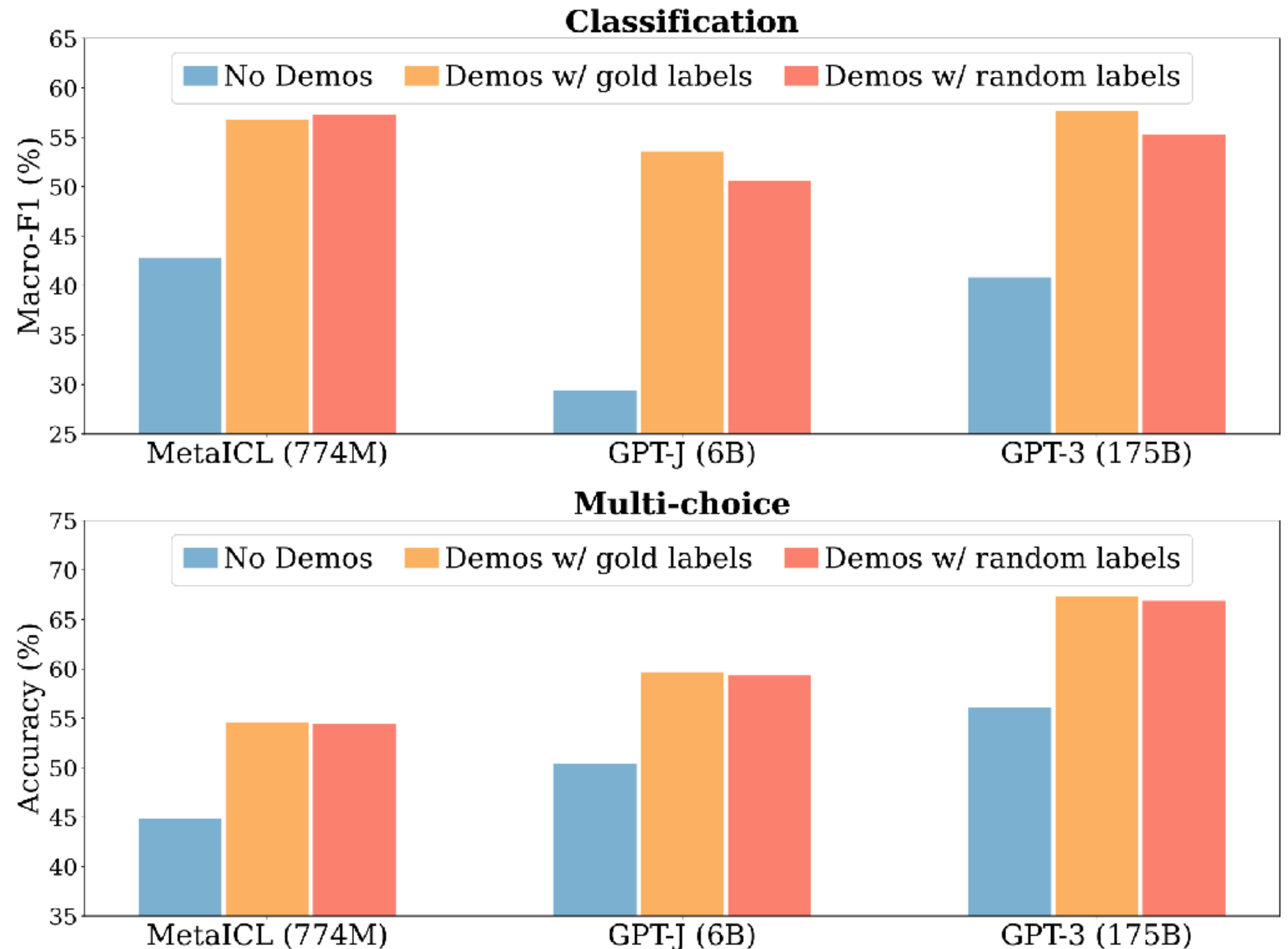
Context length: 400

Context length: 1400

- ICL is sufficiently powerful that models can rearrange their *representations* of a task to match the true structure of the task *with no gradient updates*!
- More on this when we get to interpretability.

Why do labeled examples help?

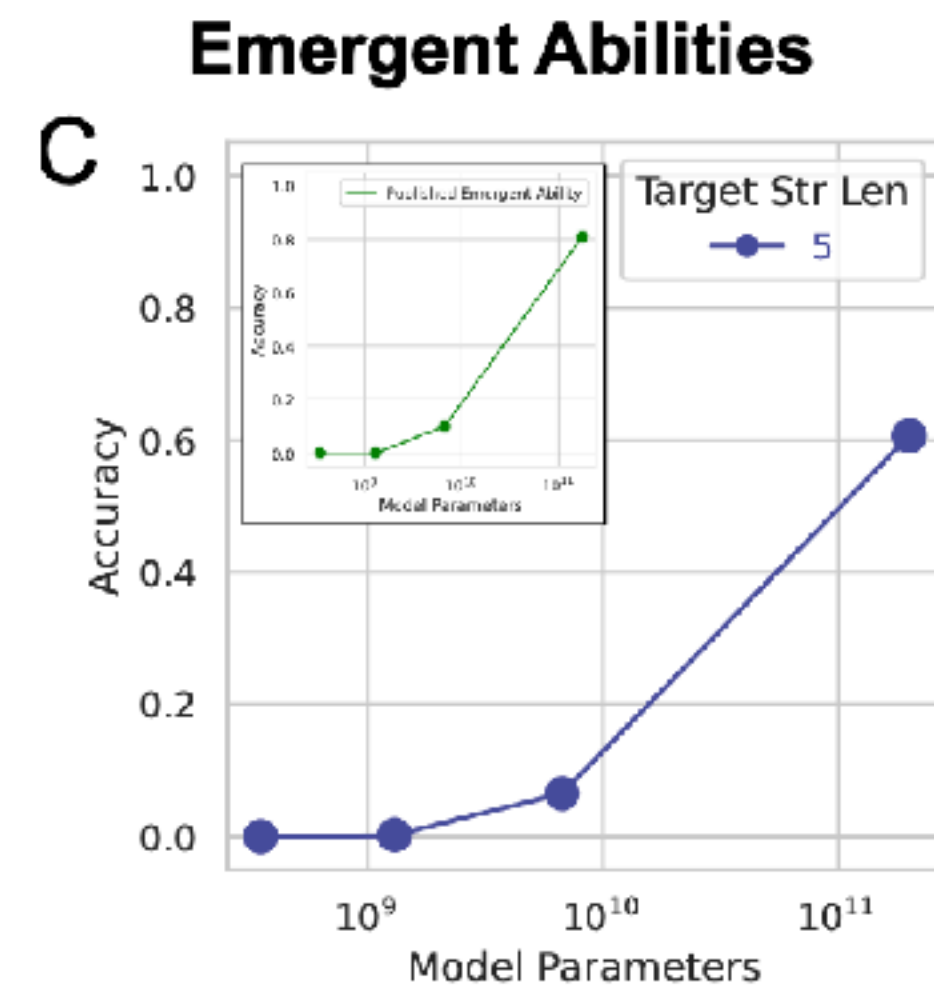
- We might reasonably assume the model is learning something about the task from labeled examples.
- But even if you *randomize the labels*, the model performs almost just as well!
 - The *form* of the demonstrations might be more important than the content.



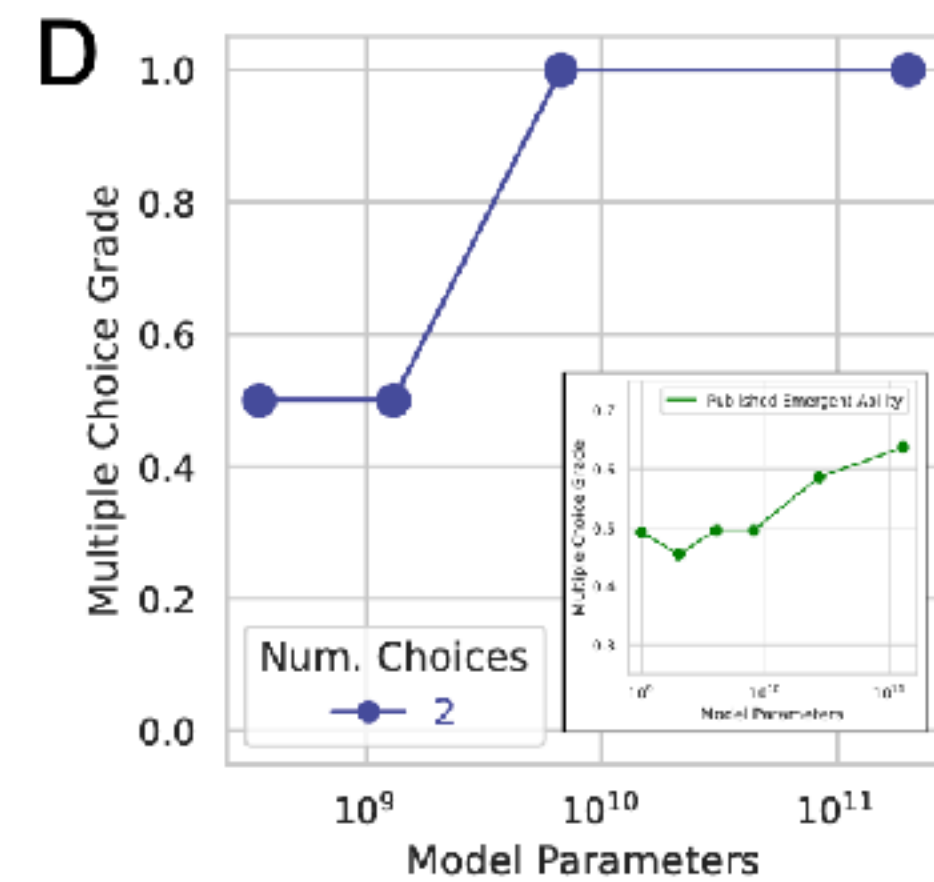
[Min et al., 2022]

Emergence may be an evaluation artifact.

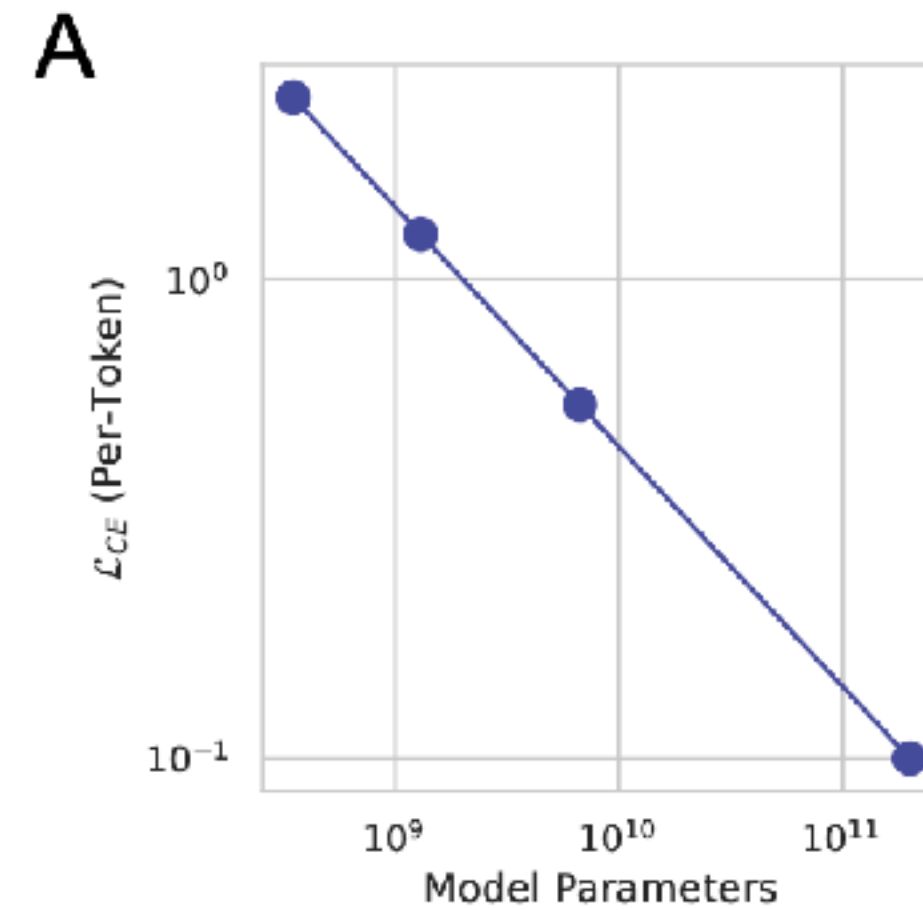
[Schaeffer et al., 2023]



**Nonlinearly
score
LLM outputs**

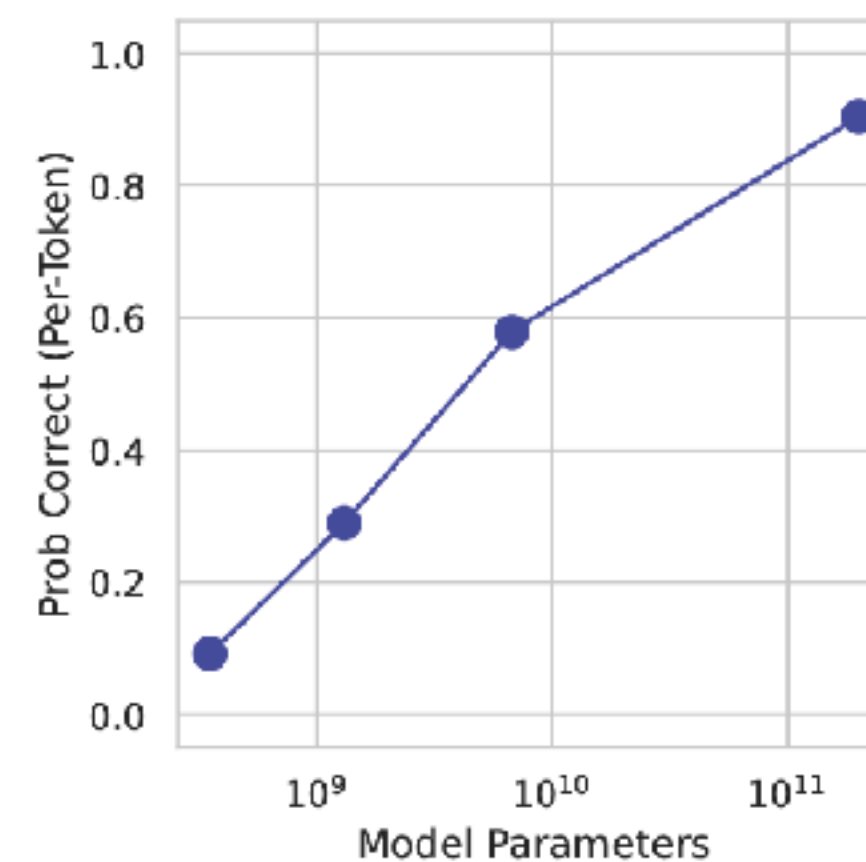


**Discontinuously
score
LLM outputs**



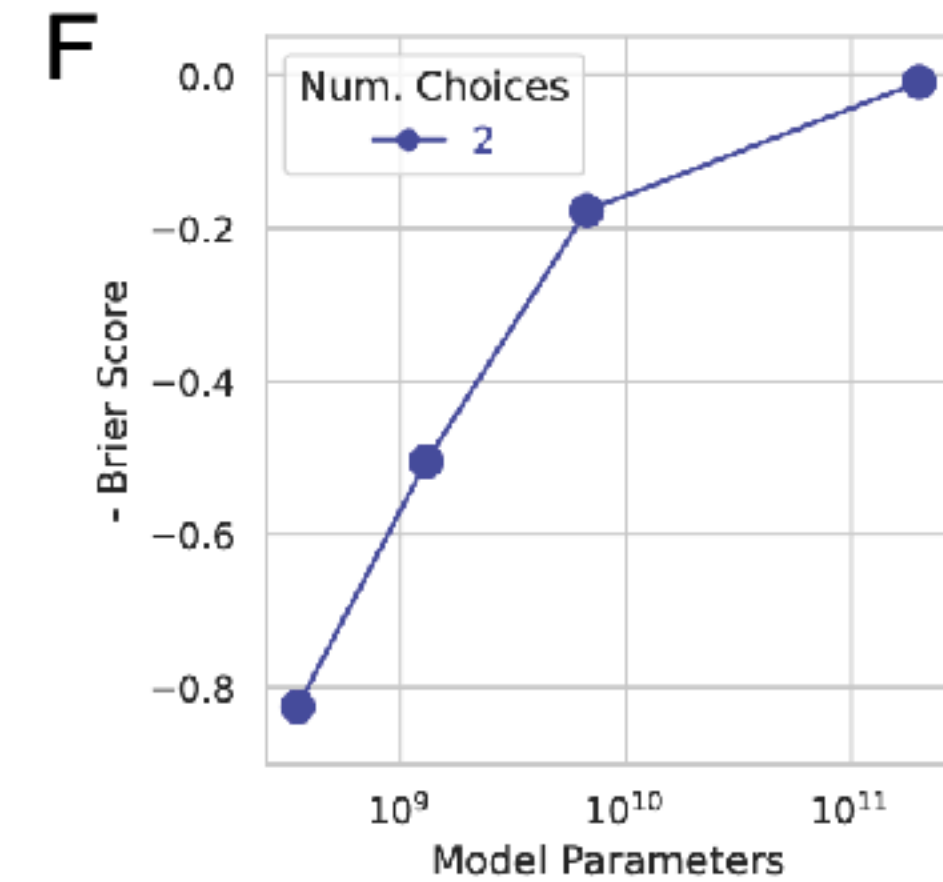
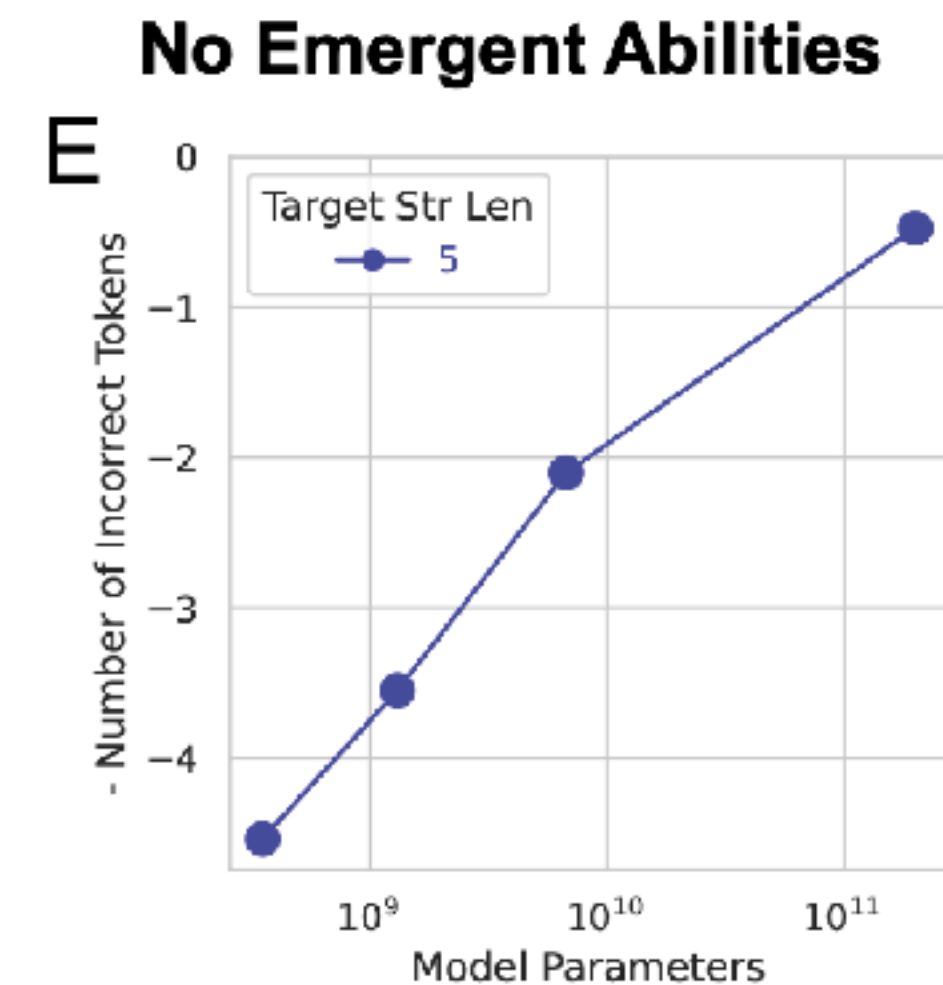
B

$$p(\text{single token correct}) = \exp(-\mathcal{L}_{CE}(N))$$



**Linearly
score
LLM outputs**

**Continuously
score
LLM outputs**



Performance isn't everything.

- **Size:** bigger models take a lot of time and GPUs to train, and a lot of memory to store.
- **Energy usage:** larger models/more tokens require more energy, and thus usually lead to more CO2 emissions
 - Water usage (for cooling data center servers) has also been controversial recently
- **Fairness/safety:** even when task performance is good, language models can often pick up on stereotypes, have systematically lower performance when handling certain dialects or information from certain demographic groups, and/or engage in “reward hacking”